

Machine Learning

Part 1: Regression

Md. Shaon Khan

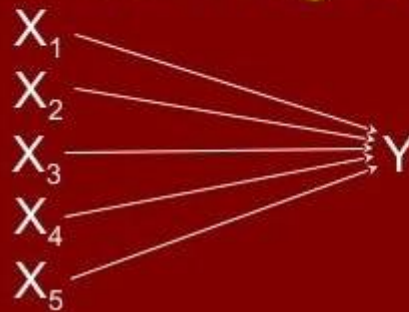
B.Sc. in Information Technology — IIT, Jahangirnagar University

Linear Regression

Single predictor $X \longrightarrow Y$

Multiple Linear Regression

Multiple
predictors



Contents

PART 1 — REGRESSION	7
Chapter 1 — Simple Linear Regression	8
1. Introduction	8
2. Why This Algorithm Was Developed	8
3. Real-life Motivation	8
4. Intuition	8
5. Working Principle	8
6. Mathematical Backend	9
7. Formula	9
8. Formula Derivation	9
9. Meaning of Every Symbol.....	9
10. Step-by-Step Formula Explanation (Full Backend Workflow).....	10
11. Numerical Example	10
12. Visualization	10
13. Hyperparameters	11
14. Scikit-learn Import	11
15. Python Example	12
16. Prediction Process.....	13
17. Advantages.....	13
18. Disadvantages.....	13
19. Applications.....	13
20. Comparison with Previous Algorithm.....	14
21. Common Mistakes	14
22. Interview Questions	14
23. Summary	14
24. Complete Mathematical Implementation (Full Manual Walkthrough)	15
Chapter 2 — Multiple Linear Regression	23
1. Introduction	23
2. Why This Algorithm Was Developed	23
3. Real-life Motivation	23
4. Intuition	23
5. Working Principle	23
6. Mathematical Backend	23
7. Formula	24

8. Formula Derivation	24
9. Meaning of Every Symbol.....	24
10. Step-by-Step Formula Explanation (Full Backend Workflow).....	24
11. Numerical Example	25
12. Visualization	25
13. Hyperparameters	25
14. Scikit-learn Import	27
15. Python Example	27
16. Prediction Process	28
17. Advantages	28
18. Disadvantages.....	29
19. Applications	29
20. Comparison with Previous Algorithm	29
21. Common Mistakes	30
22. Interview Questions	30
23. Summary	30
24. Complete Mathematical Implementation (Full Manual Walkthrough)	30
Chapter 3 — Ordinary Least Squares (OLS)	38
1. Introduction	38
2. Why This Algorithm Was Developed	38
3. Real-life Motivation	38
4. Intuition	38
5. Working Principle	38
6. Mathematical Backend	39
7. Formula	39
8. Formula Derivation — Step-by-step Mechanism	39
9. Meaning of Every Symbol.....	39
10. Step-by-Step Formula Explanation	39
11. Numerical Example	40
12. Visualization	40
13. Hyperparameters	40
14. Scikit-learn Import	40
15. Python Example	41
16. Prediction Process	41
17. Advantages	41

18. Disadvantages.....	41
19. Applications.....	42
20. Comparison with Previous Algorithm.....	42
21. Common Mistakes	42
22. Interview Questions	42
23. Summary	42
24. Complete Mathematical Implementation (Full Manual Walkthrough)	43
Chapter 4 — Ridge Regression	46
1. Introduction	46
2. Why This Algorithm Was Developed	46
3. Real-life Motivation	46
4. Intuition	46
5. Working Principle	46
6. Mathematical Backend	46
7. Formula	47
8. Formula Derivation	47
9. Meaning of Every Symbol.....	47
10. Step-by-Step Formula Explanation	47
11. Numerical Example	47
12. Visualization	48
13. Hyperparameters	48
14. Scikit-learn Import	48
15. Python Example	49
16. Prediction Process.....	49
17. Advantages.....	49
18. Disadvantages.....	49
19. Applications.....	49
20. Comparison with Previous Algorithm.....	50
21. Common Mistakes	50
22. Interview Questions	50
23. Summary	50
24. Complete Mathematical Implementation (Full Manual Walkthrough)	51
Chapter 5 — Lasso Regression.....	53
1. Introduction	53
2. Why This Algorithm Was Developed	53

3. Real-life Motivation	53
4. Intuition	53
5. Working Principle	53
6. Mathematical Backend	53
7. Formula	54
8. Formula Derivation	54
9. Meaning of Every Symbol.....	54
10. Step-by-Step Formula Explanation	54
11. Numerical Example	54
12. Visualization	55
13. Hyperparameters	55
14. Scikit-learn Import	55
15. Python Example	56
16. Prediction Process	56
17. Advantages	56
18. Disadvantages.....	56
19. Applications	56
20. Comparison with Previous Algorithm	57
21. Common Mistakes	57
22. Interview Questions	57
23. Summary	57
24. Complete Mathematical Implementation (Full Manual Walkthrough)	58
Chapter 6 — Elastic Net Regression	65
1. Introduction	65
2. Why This Algorithm Was Developed	65
3. Real-life Motivation	65
4. Intuition	65
5. Working Principle	65
6. Mathematical Backend	65
7. Formula	65
8. Formula Derivation	66
9. Meaning of Every Symbol.....	66
10. Step-by-Step Formula Explanation	66
11. Numerical Example	66
12. Visualization	67

13. Hyperparameters	67
14. Scikit-learn Import	67
15. Python Example	68
16. Prediction Process	68
17. Advantages	68
18. Disadvantages	68
19. Applications	68
20. Comparison with Previous Algorithm	69
21. Common Mistakes	69
22. Interview Questions	69
23. Summary	69
24. Complete Mathematical Implementation (Full Manual Walkthrough)	70

PART 1 — REGRESSION

Part 1 এই বইয়ের ভিত্তি অধ্যায়। Regression হলো Supervised Machine Learning-এর একটি শাখা, যেখানে Model একটি Continuous (সংখ্যাগত) মান ভবিষ্যদ্বাণী করতে শেখে — যেমন বাড়ির দাম, শেয়ারের মূল্য, তাপমাত্রা, বা বিক্রয়ের পরিমাণ। এটি Classification থেকে আলাদা, কারণ Classification-এ আউটপুট একটি নির্দিষ্ট Category (যেমন Yes/No বা Spam/Not-Spam) হয়, কিন্তু Regression-এ আউটপুট একটি নির্দিষ্ট সংখ্যা হয়।

Regression গুরুত্বপূর্ণ কারণ এটি বাস্তব জগতের অসংখ্য সমস্যায় সরাসরি প্রয়োগযোগ্য — ফিন্যান্স, রিয়েল এস্টেট, স্বাস্থ্যসেবা, বিজ্ঞাপন, ও ইঞ্জিনিয়ারিং সব জায়গায় এর ব্যবহার আছে। এই Part-এ আমরা সবচেয়ে সরল মডেল (Simple Linear Regression) থেকে শুরু করে ধীরে ধীরে জটিলতর মডেলের (Ridge, Lasso, Elastic Net) দিকে এগিয়ে যাব।

এই Part-এর ছয়টি Chapter একটি স্বাভাবিক শিক্ষণ-পথ অনুসরণ করে:

- Chapter 1 — Simple Linear Regression: একটি মাত্র Feature দিয়ে সরলরৈখিক সম্পর্ক মডেল করা।
- Chapter 2 — Multiple Linear Regression: একাধিক Feature ব্যবহার করে বাস্তবসম্মত সমস্যার সমাধান।
- Chapter 3 — Ordinary Least Squares (OLS): Gradient Descent ছাড়াই সরাসরি গাণিতিক সমাধান বের করার পদ্ধতি।
- Chapter 4 — Ridge Regression: L2 Regularization ব্যবহার করে Overfitting নিয়ন্ত্রণ।
- Chapter 5 — Lasso Regression: L1 Regularization ব্যবহার করে স্বয়ংক্রিয় Feature Selection।
- Chapter 6 — Elastic Net Regression: Ridge ও Lasso-এর সমন্বিত শক্তি।

প্রতিটি Chapter পূর্ববর্তী Chapter-এর ওপর ভিত্তি করে গড়ে উঠবে, যাতে বইয়ের শেষে গিয়ে আপনি সম্পূর্ণ Regression Family-এর একটি প্রফেশনাল-স্তরের ধারণা লাভ করতে পারেন। Part 2-এ আমরা Regression থেকে Classification-এ প্রবেশ করব, যেখানে Continuous মানের বদলে Discrete Category ভবিষ্যদ্বাণী করা শেখানো হবে।

Chapter 1 — Simple Linear Regression

1. Introduction

Simple Linear Regression (সিম্পল লিনিয়ার রিগ্রেশন) হলো Machine Learning-এর সবচেয়ে মৌলিক ও প্রথম শেখা Algorithm। এটি একটি ইনপুট ফিচার (x) এবং একটি আউটপুট টার্গেট (y)-এর মধ্যে সরলরৈখিক (linear) সম্পর্ক তৈরি করে ভবিষ্যদ্বাণী করে। এটি একটি Supervised Learning Algorithm এবং Regression সমস্যার (Continuous Value Prediction) জন্য ব্যবহৃত হয়।

এই অধ্যায়ে আমরা দেখব কীভাবে একটি সরল রেখা (straight line) দিয়ে ডেটার প্যাটার্ন প্রকাশ করা যায়, এই রেখার সবচেয়ে ভালো Weight ও Bias কীভাবে গাণিতিকভাবে খুঁজে বের করা হয়, এবং কীভাবে Scikit-Learn ব্যবহার করে এটি বাস্তবে প্রয়োগ করা হয়।

2. Why This Algorithm Was Developed

উনিশ শতকের শুরুর দিকে পরিসংখ্যানবিদরা (statisticians) লক্ষ্য করেন যে বাস্তব জগতের অনেক ঘটনার মধ্যে একটি আনুমানিক সরলরৈখিক সম্পর্ক থাকে — যেমন উচ্চতা বনাম ওজন, বা চাষের সার বনাম ফলন। এই সম্পর্কগুলো নির্ভুলভাবে বর্ণনা করার এবং ভবিষ্যদ্বাণী করার জন্য একটি সহজ, গাণিতিকভাবে ব্যাখ্যাযোগ্য (interpretable) পদ্ধতির প্রয়োজন ছিল। Linear Regression ঠিক এই প্রয়োজন থেকেই তৈরি হয়েছিল — একটি ন্যূনতম জটিলতার Model যা ডেটার মধ্যকার প্রবণতা (trend) ধরতে পারে এবং একই সাথে সহজে ব্যাখ্যা করা যায়।

3. Real-life Motivation

বাস্তব জীবনে এমন অনেক সমস্যা আছে যেখানে দুটি ভেরিয়েবলের মধ্যে একটি Approximate Linear সম্পর্ক বিদ্যমান — যেমন পড়াশোনার সময় বনাম পরীক্ষার নম্বর, বাসার আয়তন বনাম মূল্য, বিজ্ঞাপনে ব্যয় বনাম বিক্রয়। Simple Linear Regression এই সম্পর্কটি গাণিতিকভাবে মডেল করে ভবিষ্যদ্বাণী করতে সাহায্য করে।

4. Intuition

কল্পনা করুন একটি স্ক্যাটার প্লটে (scatter plot) কিছু বিন্দু ছড়িয়ে আছে। Simple Linear Regression-এর কাজ হলো এমন একটি সরলরেখা আঁকা, যেটি এই বিন্দুগুলোর যতটা সম্ভব কাছ দিয়ে যায় — অর্থাৎ রেখা এবং প্রতিটি বিন্দুর মধ্যকার দূরত্ব (error) সর্বনিম্ন থাকে। এই রেখাটিই ভবিষ্যতে নতুন x -এর জন্য y অনুমান করতে ব্যবহৃত হয়।

5. Working Principle

মডেলটি ধাপে ধাপে শেখে: প্রথমে Weight (w) এবং Bias (b)-কে দৈবভাবে (বা শূন্য দিয়ে) শুরু করা হয়, তারপর বর্তমান w ও b ব্যবহার করে প্রেডিকশন করা হয়, প্রেডিকশনের ভুল পরিমাপ করা হয়, এবং সেই

ভুলের ভিত্তিতে w ও b ধীরে ধীরে আপডেট করা হয় – যতক্ষণ না ভুল সর্বনিম্ন হয়ে যায়। এই সম্পূর্ণ প্রক্রিয়াটি নিচের ৬ নম্বর সেকশনে বিস্তারিতভাবে ব্যাখ্যা করা হয়েছে।

6. Mathematical Backend

সিম্পল লিনিয়ার রিগ্রেশন মূলত একটি ইনপুট ফিচার (x) এবং একটি আউটপুট টার্গেটের (y) মধ্যে সরলরৈখিক সম্পর্ক তৈরি করে।

7. Formula

$$\hat{y} = wX + b$$

8. Formula Derivation

মডেলের প্রেডিকশন আসল ডেটার তুলনায় কতটা ভুল করছে, তা পরিমাপ করার জন্য Mean Squared Error (MSE) কস্ট ফাংশন ব্যবহার করা হয়:

$$J(w,b) = (1/n) \sum_i (y_i - \hat{y}_i)^2$$

যেহেতু $\hat{y}_i = wX_i + b$, তাই সমীকরণটি দাঁড়ায়:

$$J(w,b) = (1/n) \sum_i (wX_i + b - y_i)^2$$

গ্রাডিয়েন্ট ডিসেন্ট পদ্ধতিতে J -এর মান সর্বনিম্ন করার জন্য w ও b -এর সাপেক্ষে আংশিক অন্তরীকরণ (chain rule ব্যবহার করে) করা হয়:

$$\partial J / \partial w = (2/n) \sum_i (wX_i + b - y_i) \cdot X_i$$

$$\partial J / \partial b = (2/n) \sum_i (wX_i + b - y_i)$$

এখানে $(wX_i + b - y_i)$ অংশটি হলো মূলত প্রেডিকশন এবং আসল মানের মধ্যকার Error। প্রতিটি ইটারেশনে এই গ্রাডিয়েন্টের ওপর ভিত্তি করে Learning Rate (α) গুণ করে w ও b আপডেট করা হয়:

$$w := w - \alpha \cdot \partial J / \partial w \quad b := b - \alpha \cdot \partial J / \partial b$$

9. Meaning of Every Symbol

Symbol	Meaning
$\hat{y}$ (y-hat)	মডেলের প্রেডিক্ট করা মান (Predicted Value)
x	ইনপুট ফিচার (Independent Variable)
w (Weight/Slope)	রেখাটির ঢাল – x এক ইউনিট বাড়লে y কতটুকু বাড়বে/কমবে
b (Bias/Intercept)	$x = 0$ হলে y -এর মান কত হবে তা নির্দেশ করে
n	মোট ডেটা স্যাম্পলের সংখ্যা

Symbol	Meaning
y_i	i-তম Sample-এর আসল মান (Actual Value)
$\hat{y}_i$	i-তম Sample-এর প্রেডিক্ট করা মান
α (Alpha)	Learning Rate — মডেল কতটা বড় স্টেপে গ্লোবাল মিনিমামের দিকে নামবে

10. Step-by-Step Formula Explanation (Full Backend Workflow)

- Initialization: শুরুতেই $w = 0$ এবং $b = 0$ ধরে নেওয়া হয়।
- Forward Pass (Prediction): বর্তমান w ও b দিয়ে পুরো ডেটাসেটের জন্য $\hat{y} = wX + b$ সূত্র ব্যবহার করে প্রেডিকশন হিসাব করা হয়।
- Compute Cost: প্রেডিকশনের সাথে আসল মানের তুলনা করে MSE সূত্রের মাধ্যমে বর্তমান কস্ট (J) মাপা হয়।
- Backward Pass (Gradient Computation): আংশিক অন্তরীকরণের সূত্র ব্যবহার করে $\partial J / \partial w$ এবং $\partial J / \partial b$ নির্ণয় করা হয়।
- Update Parameters: গ্রাডিয়েন্ট ব্যবহার করে Learning Rate গুণ করে w ও b -এর নতুন মান আপডেট করা হয়।
- Iteration: ২ থেকে ৫ নম্বর ধাপ হাজার হাজার বার (যেমন 1,500 বার) লুপে চলতে থাকে; প্রতিবার কস্ট কমতে থাকে এবং একপর্যায়ে স্থির হয়ে যায়।

11. Numerical Example

ধরা যাক আমাদের একটি ছোট ডেটাসেট আছে যেখানে x = পড়াশোনার ঘণ্টা এবং y = পরীক্ষার নম্বর:

X (Study Hours)	y (Marks)
1	2
2	4
3	6

এখানে সহজেই বোঝা যায় প্রতি ঘণ্টায় নম্বর ২ করে বাড়ছে, অর্থাৎ $w = 2$ এবং $b = 0$ হলে $\hat{y} = 2x$ সূত্রটি প্রতিটি ডেটা পয়েন্টের জন্য নিখুঁতভাবে মিলে যায় ($J(w,b) = 0$)। Gradient Descent যদি এই ডেটাসেটে চালানো হয়, তাহলে বহু ইটারেশনের পর w ও b ধীরে ধীরে এই মানের দিকে কনভারজ করবে। Chapter 3-এ আমরা দেখব কীভাবে OLS-এর Closed-form সূত্র ব্যবহার করে এই একই ফলাফল একবারেই (কোনো ইটারেশন ছাড়া) বের করা যায়।

12. Visualization

লিনিয়ার রিগ্রেশনের কস্ট ফাংশনের গ্রাফটি দেখতে একটি বাটি বা বোলের (Bowl) মতো হয়, যাকে গাণিতিক ভাষায় Convex Function বলা হয়। গ্রাডিয়েন্ট ডিসেন্ট পাহাড়ের চূড়া থেকে আস্তে আস্তে পা ফেলে বাটির একদম তলানিতে পৌঁছানোর চেষ্টা করে, যেখানে ভুলের পরিমাণ সবচেয়ে কম।

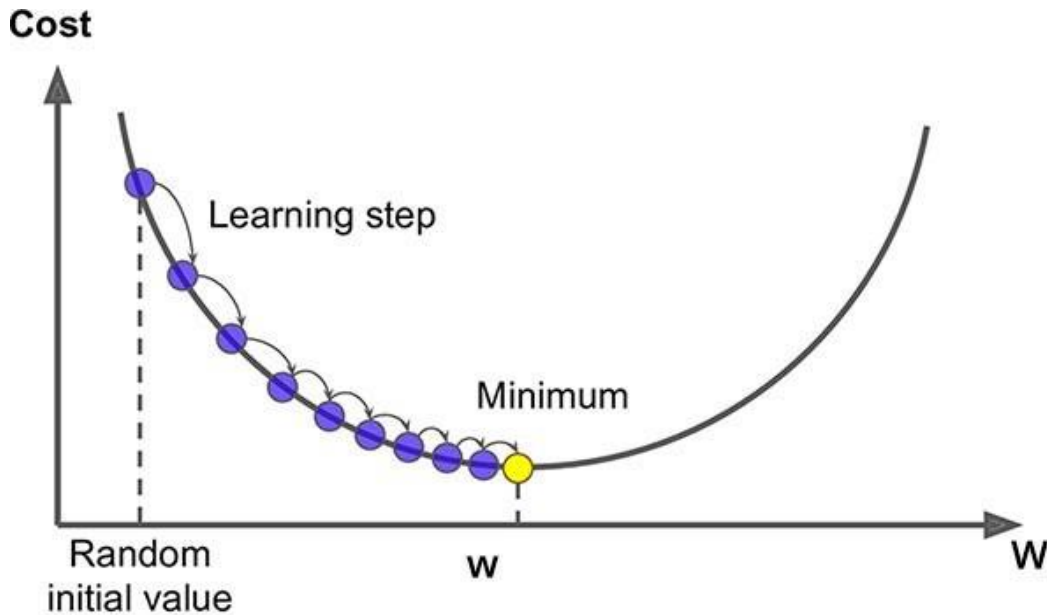


Figure 1.1 — Convex (bowl-shaped) cost surface of Linear Regression

টেক্সট আকারে ওয়ার্কফ্লো ডায়াগ্রাম:

Init(w,b) → Predict($\hat{y}=wX+b$) → Cost(J) → Gradient($\partial J/\partial w, \partial J/\partial b$) → Update(w,b)
→ repeat

13. Hyperparameters

মূল Simple Linear Regression Model-এর গাণিতিক ফর্মে সরাসরি কোনো Hyperparameter নেই; তবে Gradient Descent দিয়ে বাস্তবায়ন করার সময় নিচের সেটিংসগুলো গুরুত্বপূর্ণ হয়ে ওঠে।

Hyperparameter	ভূমিকা
Learning Rate (α)	প্রতিটি স্টেপে w ও b কতটা পরিবর্তিত হবে তা নিয়ন্ত্রণ করে
Number of Iterations	গ্রাডিয়েন্ট ডিসেন্ট কতবার চলবে, কনভার্জেন্স নিশ্চিত করতে ব্যবহৃত হয়
fit_intercept (sklearn)	b (intercept) শেখা হবে কিনা তা নির্ধারণ করে, ডিফল্ট True

14. Scikit-learn Import

```
from sklearn.linear_model import LinearRegression, SGDRegressor
```

15. Python Example

From Scratch (No Scikit-Learn) — নিচের কোডে গ্র্যাডিয়েন্ট ডিসেন্ট ব্যবহার করে ম্যানুয়ালি Weight ও Bias শেখানো হয়েছে:

```
1 import numpy as np
2
3 def make_prediction(X, w, b):
4     return w * X + b
5
6 def cost_function(X, y, w, b):
7     n = len(y)
8     y_pred = make_prediction(X, w, b)
9     return (1 / n) * np.sum((y_pred - y) ** 2)
10
11 def calculate_gradient(X, y, w, b):
12     n = len(y)
13     y_pred = make_prediction(X, w, b)
14     dw = (2 / n) * np.sum((y_pred - y) * X)
15     db = (2 / n) * np.sum((y_pred - y))
16     return dw, db
17
18 def gradient_descent(X, y, w, b, lr, epochs):
19     for i in range(epochs):
20         dw, db = calculate_gradient(X, y, w, b)
21         w = w - lr * dw
22         b = b - lr * db
23         if i % 100 == 0:
24             print("epoch:", i, "cost:", cost_function(X, y, w, b))
25     return w, b
26
27 X = np.array([1, 2, 3, 4, 5])
28 y = np.array([2, 4, 6, 8, 10])
29
30 w, b = gradient_descent(X, y, 0, 0, 0.01, 1000)
31
32 print("Final w:", w)
33 print("Final b:", b)
```

Scikit-Learn ব্যবহার করে (Closed-form OLS solution):

```
1 import numpy as np
2 from sklearn.linear_model import LinearRegression, SGDRegressor
3 from sklearn.preprocessing import StandardScaler
4
5 X = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
6 y = np.array([2, 4, 6, 8, 10])
7
8 lr_model = LinearRegression()
9 lr_model.fit(X, y)
10
11 print("Linear Regression")
12 print("w:", lr_model.coef_[0])
13 print("b:", lr_model.intercept_)
14
15 print("Pred:", lr_model.predict(X))
```

SGDRegressor ব্যবহার করে (Gradient Descent-based solution):

```
18 scaler = StandardScaler()
19 X_scaled = scaler.fit_transform(X)
20
21 sgd_model = SGDRegressor(
22     learning_rate="constant",
23     eta0=0.01,
24     max_iter=1000,
25     tol=1e-3
26 )
27
28 sgd_model.fit(X_scaled, y)
29
30 print("\nSGD Regression")
31 print("w:", sgd_model.coef_[0])
32 print("b:", sgd_model.intercept_[0])
33 print("Pred:", sgd_model.predict(X_scaled))
```

16. Prediction Process

একবার Training সম্পন্ন হয়ে w ও b -এর চূড়ান্ত মান পাওয়া গেলে, নতুন যেকোনো Input X -এর জন্য শুধু $\hat{y} = wX + b$ সূত্রে বসিয়ে সরাসরি প্রেডিকশন পাওয়া যায় — এই ধাপে কোনো Iteration বা পুনরায় Training-এর প্রয়োজন হয় না।

17. Advantages

- সহজ, দ্রুত এবং Interpretable — Coefficient দেখেই সম্পর্ক বোঝা যায়।
- কম কম্পিউটেশনাল রিসোর্সে চলে।
- Closed-form (OLS) সমাধান বিদ্যমান, তাই দ্রুত ও নির্ভুল ফলাফল পাওয়া যায়।
- MSE সবসময় Non-negative এবং Convex, তাই Gradient Descent একটি Unique Global Minimum খুঁজে পায়।

18. Disadvantages

- শুধু Linear সম্পর্কের জন্য কার্যকর — Non-linear প্যাটার্নে খারাপ পারফর্ম করে।
- Outlier-এর প্রতি অত্যন্ত sensitive; একটি বড় Outlier পুরো রেখার Slope বদলে দিতে পারে।
- একাধিক ফিচার থাকলে (Multiple Feature) Multicollinearity সমস্যা তৈরি করতে পারে।

19. Applications

- হাউজিং মার্কেটে শুধুমাত্র বাসার আয়তন (sq ft) দিয়ে আনুমানিক মূল্য নির্ধারণ।
- কোনো প্রতিষ্ঠানের বিজ্ঞাপন ব্যয় থেকে বিক্রয় পরিমাণ পূর্বাভাস।
- পড়াশোনার ঘণ্টা থেকে পরীক্ষার নম্বর অনুমান করা।

20. Comparison with Previous Algorithm

যেহেতু Simple Linear Regression এই বইয়ের প্রথম Algorithm, তাই তুলনা করার মতো পূর্ববর্তী কোনো Algorithm নেই। তবে এটি Multiple Linear Regression-এর ভিত্তি — Chapter 2-এ আমরা দেখব কীভাবে একাধিক Feature ব্যবহার করে এই একই ধারণাকে সম্প্রসারিত করা হয়।

ASSUMPTIONS

Linearity: X ও y -এর মধ্যে সম্পর্ক সরলরৈখিক হতে হবে। Independence: Observation-গুলো পরস্পর স্বাধীন হতে হবে। Homoscedasticity: Residual-এর Variance সব জায়গায় প্রায় সমান থাকতে হবে। Normality of Residuals: Error term-গুলো Normal Distribution অনুসরণ করা উচিত।

21. Common Mistakes

- Outlier বাদ না দিয়ে সরাসরি Model Train করা — এতে Slope (w) মারাত্মকভাবে প্রভাবিত হতে পারে।
- Feature ও Target-এর মধ্যে সম্পর্ক Non-linear হলেও জোর করে Linear Model ব্যবহার করা।
- Learning Rate (α) অত্যধিক বড় নেওয়া, যার ফলে Gradient Descent Diverge করে।

22. Interview Questions

Q: কেন আমরা MSE-তে error-কে square করি, absolute value নিই না কেন?

A: Square করলে ফাংশনটি Differentiable এবং Convex হয়, ফলে Gradient Descent সহজে কাজ করতে পারে; এছাড়া বড় error-কে বেশি penalize করা যায়।

Q: OLS এবং Gradient Descent-এর মধ্যে পার্থক্য কী?

A: OLS একটি Closed-form, একবারেই সমাধানকারী পদ্ধতি; Gradient Descent ধাপে ধাপে Iterative ভাবে Optimal মান খুঁজে বের করে। বিস্তারিত আলোচনা Chapter 3-এ।

23. Summary

Category	Key Points
Key Formula	$\hat{y} = wX + b$; $J(w,b) = (1/n)\sum (y_i - \hat{y}_i)^2$
Gradient	$\partial J / \partial w = (2/n)\sum (\text{error} \cdot X)$; $\partial J / \partial b = (2/n)\sum (\text{error})$
Update Rule	$w := w - \alpha \cdot \partial J / \partial w$; $b := b - \alpha \cdot \partial J / \partial b$
Cost Surface	Convex (bowl-shaped) $\rightarrow$ guarantees a global minimum
Exam Focus	Derive gradients via chain rule; explain why MSE is convex
Interview Focus	Assumptions of linear regression; OLS vs Gradient Descent
PRACTICAL INSIGHT	

Category	Key Points
	Forward Pass → Compute Cost → Backward Pass → Update — এই চক্রটিই প্রায় সকল Gradient-Based Machine Learning ও Deep Learning Model-এর মূল ভিত্তি। এই ৪টি ধাপ ভালোভাবে আত্মস্থ করলে Neural Network-এর Backpropagation বুঝতেও সুবিধা হয়।

চ্যাপ্টারের শেষে: পরবর্তী অধ্যায়ে আমরা একই ধারণাকে একাধিক Feature-এ সম্প্রসারিত করে Multiple Linear Regression শিখব — যেখানে একটির বদলে অনেকগুলো ইনপুট ভেরিয়েবল ব্যবহার করে একইসাথে অনেক বেশি বাস্তবসম্মত সমস্যার সমাধান করা যায়, যেমন একাধিক ফ্যাক্টরের ওপর ভিত্তি করে বাড়ির দাম অনুমান করা।

24. Complete Mathematical Implementation (Full Manual Walkthrough)

এই সেকশনে Simple Linear Regression মডেলটি সম্পূর্ণভাবে হাতে-কলমে (Manual Calculation) দেখানো হবে — Dataset থেকে শুরু করে Final Equation ও নতুন Prediction পর্যন্ত, যাতে একজন Beginner Calculator নিয়ে প্রতিটি ধাপ নিজে Verify করতে পারেন।

Dataset

হাতে গণনার সুবিধার জন্য একটি খুব ছোট Dataset নেওয়া হলো, যেখানে ৪টি Sample আছে:

Sample (i)	X (Feature)	y (Actual)
1	1	2
2	2	4
3	3	5
4	4	4

Initialization

Training শুরু করার আগে Weight ও Bias উভয়কেই শূন্য দিয়ে শুরু করা হয়, এবং Learning Rate (α) একটি ছোট সংখ্যা হিসেবে বেছে নেওয়া হয় যাতে Update খুব বড় লাফ না দেয়:

$$w = 0, \quad b = 0, \quad \alpha = 0.05, \quad n = 4$$

Forward Pass ও Formula

প্রতিটি Sample-এর জন্য প্রেডিকশন বের করার Formula:

$$\hat{y}_i = w \cdot X_i + b$$

প্রেডিকশনের ভুল পরিমাপ করার জন্য Mean Squared Error (MSE) ব্যবহার করা হয়:

$$J(w,b) = (1/n) \sum_i (\hat{y}_i - y_i)^2$$

এই J-কে সর্বনিম্ন করার জন্য w ও b-এর সাপেক্ষে Partial Derivative (Chain Rule ব্যবহার করে) বের করা হয় – কারণ Gradient হলো সেই দিক, যেরূপে J সবচেয়ে দ্রুত বাড়ে; তাই Gradient-এর বিপরীত দিকে গেলে J সবচেয়ে দ্রুত কমে:

$$\partial J / \partial w = (2/n) \sum_i (\hat{y}_i - y_i) \cdot X_i \quad \partial J / \partial b = (2/n) \sum_i (\hat{y}_i - y_i)$$

প্রতিটি Iteration-এ Gradient বের করে Learning Rate দিয়ে গুণ করে w ও b থেকে বিয়োগ করা হয়:

$$w := w - \alpha \cdot \partial J / \partial w \quad b := b - \alpha \cdot \partial J / \partial b$$

Iteration-by-Iteration Manual Calculation

Iteration 1 (শুরু w = 0.0000, b = 0.0000)

প্রথমে বর্তমান w = 0.0000 এবং b = 0.0000 বসিয়ে প্রতিটি Sample-এর জন্য Prediction বের করা হলো:

$$\begin{aligned} \hat{y}_1 &= 0.0000 \times 1 + 0.0000 = 0.0000 & \hat{y}_2 &= 0.0000 \times 2 + 0.0000 = 0.0000 & \hat{y}_3 &= \\ &0.0000 \times 3 + 0.0000 = 0.0000 & \hat{y}_4 &= 0.0000 \times 4 + 0.0000 = 0.0000 \end{aligned}$$

এবার প্রতিটি Sample-এর Error ($\hat{y} - y$) বের করা হলো:

$$\begin{aligned} e_1 &= 0.0000 - 2 = -2.0000 & e_2 &= 0.0000 - 4 = -4.0000 & e_3 &= 0.0000 - 5 = - \\ &5.0000 & e_4 &= 0.0000 - 4 = -4.0000 \end{aligned}$$

এই Error-গুলোর বর্গের গড় নিয়ে বর্তমান Iteration-এর Cost (MSE) বের করা হলো:

$$J = (1/4) \cdot [(-2.0000)^2 + (-4.0000)^2 + (-5.0000)^2 + (-4.0000)^2] = 15.2500$$

এখন Gradient বের করা হলো – প্রতিটি Error-কে সংশ্লিষ্ট x_i দিয়ে গুণ করে যোগ করে $(2/n)$ দিয়ে গুণ করলে w-এর Gradient পাওয়া যায়, আর শুধু Error-গুলো যোগ করে $(2/n)$ দিয়ে গুণ করলে b-এর Gradient পাওয়া যায়:

$$\partial J / \partial w = (2/4) \cdot [(-2.0000 \times 1) + (-4.0000 \times 2) + (-5.0000 \times 3) + (-4.0000 \times 4)] = -20.5000$$

$$\partial J / \partial b = (2/4) \cdot [(-2.0000) + (-4.0000) + (-5.0000) + (-4.0000)] = -7.5000$$

Learning Rate $\alpha = 0.05$ দিয়ে গুণ করে নতুন w ও b বের করা হলো:

$$w_{\text{new}} = 0.0000 - 0.05 \times (-20.5000) = 1.0250$$

$$b_{\text{new}} = 0.0000 - 0.05 \times (-7.5000) = 0.3750$$

Iteration 2 (শুরু w = 1.0250, b = 0.3750)

প্রথমে বর্তমান w = 1.0250 এবং b = 0.3750 বসিয়ে প্রতিটি Sample-এর জন্য Prediction বের করা হলো:

$$\hat{y}_1 = 1.0250 \times 1 + 0.3750 = 1.4000 \quad \hat{y}_2 = 1.0250 \times 2 + 0.3750 = 2.4250 \quad \hat{y}_3 = 1.0250 \times 3 + 0.3750 = 3.4500 \quad \hat{y}_4 = 1.0250 \times 4 + 0.3750 = 4.4750$$

এবার প্রতিটি Sample-এর Error ($\hat{y} - y$) বের করা হলো:

$$e_1 = 1.4000 - 2 = -0.6000 \quad e_2 = 2.4250 - 4 = -1.5750 \quad e_3 = 3.4500 - 5 = -1.5500 \quad e_4 = 4.4750 - 4 = 0.4750$$

এই Error-গুলোর বর্গের গড় নিয়ে বর্তমান Iteration-এর Cost (MSE) বের করা হলো:

$$J = (1/4) \cdot [(-0.6000)^2 + (-1.5750)^2 + (-1.5500)^2 + (0.4750)^2] = 1.3672$$

এখন Gradient বের করা হলো — প্রতিটি Error-কে সংশ্লিষ্ট x_i দিয়ে গুণ করে যোগ করে $(2/n)$ দিয়ে গুণ করলে w -এর Gradient পাওয়া যায়, আর শুধু Error-গুলো যোগ করে $(2/n)$ দিয়ে গুণ করলে b -এর Gradient পাওয়া যায়:

$$\partial J / \partial w = (2/4) \cdot [(-0.6000 \times 1) + (-1.5750 \times 2) + (-1.5500 \times 3) + (0.4750 \times 4)] = -3.2500$$

$$\partial J / \partial b = (2/4) \cdot [(-0.6000) + (-1.5750) + (-1.5500) + (0.4750)] = -1.6250$$

Learning Rate $\alpha = 0.05$ দিয়ে গুণ করে নতুন w ও b বের করা হলো:

$$w_{\text{new}} = 1.0250 - 0.05 \times (-3.2500) = 1.1875$$

$$b_{\text{new}} = 0.3750 - 0.05 \times (-1.6250) = 0.4562$$

Iteration 3 (শুরু $w = 1.1875$, $b = 0.4562$)

প্রথমে বর্তমান $w = 1.1875$ এবং $b = 0.4562$ বসিয়ে প্রতিটি Sample-এর জন্য Prediction বের করা হলো:

$$\hat{y}_1 = 1.1875 \times 1 + 0.4562 = 1.6438 \quad \hat{y}_2 = 1.1875 \times 2 + 0.4562 = 2.8312 \quad \hat{y}_3 = 1.1875 \times 3 + 0.4562 = 4.0187 \quad \hat{y}_4 = 1.1875 \times 4 + 0.4562 = 5.2062$$

এবার প্রতিটি Sample-এর Error ($\hat{y} - y$) বের করা হলো:

$$e_1 = 1.6438 - 2 = -0.3562 \quad e_2 = 2.8312 - 4 = -1.1688 \quad e_3 = 4.0187 - 5 = -0.9813 \quad e_4 = 5.2062 - 4 = 1.2062$$

এই Error-গুলোর বর্গের গড় নিয়ে বর্তমান Iteration-এর Cost (MSE) বের করা হলো:

$$J = (1/4) \cdot [(-0.3562)^2 + (-1.1688)^2 + (-0.9813)^2 + (1.2062)^2] = 0.9777$$

এখন Gradient বের করা হলো — প্রতিটি Error-কে সংশ্লিষ্ট x_i দিয়ে গুণ করে যোগ করে $(2/n)$ দিয়ে গুণ করলে w -এর Gradient পাওয়া যায়, আর শুধু Error-গুলো যোগ করে $(2/n)$ দিয়ে গুণ করলে b -এর Gradient পাওয়া যায়:

$$\partial J / \partial w = (2/4) \cdot [(-0.3562 \times 1) + (-1.1688 \times 2) + (-0.9813 \times 3) + (1.2062 \times 4)] = -0.4063$$

$$\partial J / \partial b = (2/4) \cdot [(-0.3562) + (-1.1688) + (-0.9813) + (1.2062)] = -0.6500$$

Learning Rate $\alpha = 0.05$ দিয়ে গুণ করে নতুন w ও b বের করা হলো:

$$w_{\text{new}} = 1.1875 - 0.05 \times (-0.4063) = 1.2078$$

$$b_{\text{new}} = 0.4562 - 0.05 \times (-0.6500) = 0.4888$$

Iteration 4 (শুরু $w = 1.2078$, $b = 0.4888$)

প্রথমে বর্তমান $w = 1.2078$ এবং $b = 0.4888$ বসিয়ে প্রতিটি Sample-এর জন্য Prediction বের করা হলো:

$$\hat{y}_1 = 1.2078 \times 1 + 0.4888 = 1.6966 \quad \hat{y}_2 = 1.2078 \times 2 + 0.4888 = 2.9044 \quad \hat{y}_3 = 1.2078 \times 3 + 0.4888 = 4.1122 \quad \hat{y}_4 = 1.2078 \times 4 + 0.4888 = 5.3200$$

এবার প্রতিটি Sample-এর Error ($\hat{y} - y$) বের করা হলো:

$$e_1 = 1.6966 - 2 = -0.3034 \quad e_2 = 2.9044 - 4 = -1.0956 \quad e_3 = 4.1122 - 5 = -0.8878 \quad e_4 = 5.3200 - 4 = 1.3200$$

এই Error-গুলোর বর্গের গড় নিয়ে বর্তমান Iteration-এর Cost (MSE) বের করা হলো:

$$J = (1/4) \cdot [(-0.3034)^2 + (-1.0956)^2 + (-0.8878)^2 + (1.3200)^2] = 0.9558$$

এখন Gradient বের করা হলো – প্রতিটি Error-কে সংশ্লিষ্ট x_i দিয়ে গুণ করে যোগ করে $(2/n)$ দিয়ে গুণ করলে w -এর Gradient পাওয়া যায়, আর শুধু Error-গুলো যোগ করে $(2/n)$ দিয়ে গুণ করলে b -এর Gradient পাওয়া যায়:

$$\partial J / \partial w = (2/4) \cdot [(-0.3034 \times 1) + (-1.0956 \times 2) + (-0.8878 \times 3) + (1.3200 \times 4)] = 0.0609$$

$$\partial J / \partial b = (2/4) \cdot [(-0.3034) + (-1.0956) + (-0.8878) + (1.3200)] = -0.4834$$

Learning Rate $\alpha = 0.05$ দিয়ে গুণ করে নতুন w ও b বের করা হলো:

$$w_{\text{new}} = 1.2078 - 0.05 \times (0.0609) = 1.2048$$

$$b_{\text{new}} = 0.4888 - 0.05 \times (-0.4834) = 0.5129$$

Iteration 5 (শুরু $w = 1.2048$, $b = 0.5129$)

প্রথমে বর্তমান $w = 1.2048$ এবং $b = 0.5129$ বসিয়ে প্রতিটি Sample-এর জন্য Prediction বের করা হলো:

$$\hat{y}_1 = 1.2048 \times 1 + 0.5129 = 1.7177 \quad \hat{y}_2 = 1.2048 \times 2 + 0.5129 = 2.9225 \quad \hat{y}_3 = 1.2048 \times 3 + 0.5129 = 4.1272 \quad \hat{y}_4 = 1.2048 \times 4 + 0.5129 = 5.3320$$

এবার প্রতিটি Sample-এর Error ($\hat{y} - y$) বের করা হলো:

$$e1 = 1.7177 - 2 = -0.2823 \quad e2 = 2.9225 - 4 = -1.0775 \quad e3 = 4.1272 - 5 = -0.8728 \quad e4 = 5.3320 - 4 = 1.3320$$

এই Error-গুলোর বর্গের গড় নিয়ে বর্তমান Iteration-এর Cost (MSE) বের করা হলো:

$$J = (1/4) \cdot [(-0.2823)^2 + (-1.0775)^2 + (-0.8728)^2 + (1.3320)^2] = 0.9442$$

এখন Gradient বের করা হলো – প্রতিটি Error-কে সংশ্লিষ্ট x_i দিয়ে গুণ করে যোগ করে $(2/n)$ দিয়ে গুণ করলে w -এর Gradient পাওয়া যায়, আর শুধু Error-গুলো যোগ করে $(2/n)$ দিয়ে গুণ করলে b -এর Gradient পাওয়া যায়:

$$\partial J / \partial w = (2/4) \cdot [(-0.2823 \times 1) + (-1.0775 \times 2) + (-0.8728 \times 3) + (1.3320 \times 4)] = 0.1361$$

$$\partial J / \partial b = (2/4) \cdot [(-0.2823) + (-1.0775) + (-0.8728) + (1.3320)] = -0.4503$$

Learning Rate $\alpha = 0.05$ দিয়ে গুণ করে নতুন w ও b বের করা হলো:

$$w_{\text{new}} = 1.2048 - 0.05 \times (0.1361) = 1.1980$$

$$b_{\text{new}} = 0.5129 - 0.05 \times (-0.4503) = 0.5354$$

Iteration 6 (শুরুর $w = 1.1980$, $b = 0.5354$)

প্রথমে বর্তমান $w = 1.1980$ এবং $b = 0.5354$ বসিয়ে প্রতিটি Sample-এর জন্য Prediction বের করা হলো:

$$\hat{y}1 = 1.1980 \times 1 + 0.5354 = 1.7334 \quad \hat{y}2 = 1.1980 \times 2 + 0.5354 = 2.9314 \quad \hat{y}3 = 1.1980 \times 3 + 0.5354 = 4.1293 \quad \hat{y}4 = 1.1980 \times 4 + 0.5354 = 5.3273$$

এবার প্রতিটি Sample-এর Error ($\hat{y} - y$) বের করা হলো:

$$e1 = 1.7334 - 2 = -0.2666 \quad e2 = 2.9314 - 4 = -1.0686 \quad e3 = 4.1293 - 5 = -0.8707 \quad e4 = 5.3273 - 4 = 1.3273$$

এই Error-গুলোর বর্গের গড় নিয়ে বর্তমান Iteration-এর Cost (MSE) বের করা হলো:

$$J = (1/4) \cdot [(-0.2666)^2 + (-1.0686)^2 + (-0.8707)^2 + (1.3273)^2] = 0.9332$$

এখন Gradient বের করা হলো – প্রতিটি Error-কে সংশ্লিষ্ট x_i দিয়ে গুণ করে যোগ করে $(2/n)$ দিয়ে গুণ করলে w -এর Gradient পাওয়া যায়, আর শুধু Error-গুলো যোগ করে $(2/n)$ দিয়ে গুণ করলে b -এর Gradient পাওয়া যায়:

$$\partial J / \partial w = (2/4) \cdot [(-0.2666 \times 1) + (-1.0686 \times 2) + (-0.8707 \times 3) + (1.3273 \times 4)] = 0.1466$$

$$\partial J / \partial b = (2/4) \cdot [(-0.2666) + (-1.0686) + (-0.8707) + (1.3273)] = -0.4393$$

Learning Rate $\alpha = 0.05$ দিয়ে গুণ করে নতুন w ও b বের করা হলো:

$$w_{\text{new}} = 1.1980 - 0.05 \times (0.1466) = 1.1906$$

$$b_{\text{new}} = 0.5354 - 0.05 \times (-0.4393) = 0.5574$$

Iteration 7 (শুরু w = 1.1906, b = 0.5574)

প্রথমে বর্তমান $w = 1.1906$ এবং $b = 0.5574$ বসিয়ে প্রতিটি Sample-এর জন্য Prediction বের করা হলো:

$$\hat{y}_1 = 1.1906 \times 1 + 0.5574 = 1.7480 \quad \hat{y}_2 = 1.1906 \times 2 + 0.5574 = 2.9387 \quad \hat{y}_3 = 1.1906 \times 3 + 0.5574 = 4.1293 \quad \hat{y}_4 = 1.1906 \times 4 + 0.5574 = 5.3199$$

এবার প্রতিটি Sample-এর Error ($\hat{y} - y$) বের করা হলো:

$$e_1 = 1.7480 - 2 = -0.2520 \quad e_2 = 2.9387 - 4 = -1.0613 \quad e_3 = 4.1293 - 5 = -0.8707 \quad e_4 = 5.3199 - 4 = 1.3199$$

এই Error-গুলোর বর্গের গড় নিয়ে বর্তমান Iteration-এর Cost (MSE) বের করা হলো:

$$J = (1/4) \cdot [(-0.2520)^2 + (-1.0613)^2 + (-0.8707)^2 + (1.3199)^2] = 0.9226$$

এখন Gradient বের করা হলো — প্রতিটি Error-কে সংশ্লিষ্ট x_i দিয়ে গুণ করে যোগ করে $(2/n)$ দিয়ে গুণ করলে w -এর Gradient পাওয়া যায়, আর শুধু Error-গুলো যোগ করে $(2/n)$ দিয়ে গুণ করলে b -এর Gradient পাওয়া যায়:

$$\partial J / \partial w = (2/4) \cdot [(-0.2520 \times 1) + (-1.0613 \times 2) + (-0.8707 \times 3) + (1.3199 \times 4)] = 0.1465$$

$$\partial J / \partial b = (2/4) \cdot [(-0.2520) + (-1.0613) + (-0.8707) + (1.3199)] = -0.4320$$

Learning Rate $\alpha = 0.05$ দিয়ে গুণ করে নতুন w ও b বের করা হলো:

$$w_{\text{new}} = 1.1906 - 0.05 \times (0.1465) = 1.1833$$

$$b_{\text{new}} = 0.5574 - 0.05 \times (-0.4320) = 0.5790$$

Iteration 8 (শুরু w = 1.1833, b = 0.5790)

প্রথমে বর্তমান $w = 1.1833$ এবং $b = 0.5790$ বসিয়ে প্রতিটি Sample-এর জন্য Prediction বের করা হলো:

$$\hat{y}_1 = 1.1833 \times 1 + 0.5790 = 1.7623 \quad \hat{y}_2 = 1.1833 \times 2 + 0.5790 = 2.9456 \quad \hat{y}_3 = 1.1833 \times 3 + 0.5790 = 4.1289 \quad \hat{y}_4 = 1.1833 \times 4 + 0.5790 = 5.3122$$

এবার প্রতিটি Sample-এর Error ($\hat{y} - y$) বের করা হলো:

$$e_1 = 1.7623 - 2 = -0.2377 \quad e_2 = 2.9456 - 4 = -1.0544 \quad e_3 = 4.1289 - 5 = -0.8711 \quad e_4 = 5.3122 - 4 = 1.3122$$

এই Error-গুলোর বর্গের গড় নিয়ে বর্তমান Iteration-এর Cost (MSE) বের করা হলো:

$$J = (1/4) \cdot [(-0.2377)^2 + (-1.0544)^2 + (-0.8711)^2 + (1.3122)^2] = 0.9122$$

এখন Gradient বের করা হলো – প্রতিটি Error-কে সংশ্লিষ্ট x_i দিয়ে গুণ করে যোগ করে $(2/n)$ দিয়ে গুণ করলে w -এর Gradient পাওয়া যায়, আর শুধু Error-গুলো যোগ করে $(2/n)$ দিয়ে গুণ করলে b -এর Gradient পাওয়া যায়:

$$\partial J / \partial w = (2/4) \cdot [(-0.2377 \times 1) + (-1.0544 \times 2) + (-0.8711 \times 3) + (1.3122 \times 4)] = 0.1446$$

$$\partial J / \partial b = (2/4) \cdot [(-0.2377) + (-1.0544) + (-0.8711) + (1.3122)] = -0.4255$$

Learning Rate $\alpha = 0.05$ দিয়ে গুণ করে নতুন w ও b বের করা হলো:

$$w_{\text{new}} = 1.1833 - 0.05 \times (0.1446) = 1.1761$$

$$b_{\text{new}} = 0.5790 - 0.05 \times (-0.4255) = 0.6003$$

Loss কীভাবে কমছে (Training Progress Table)

Iteration	w	b	Loss (MSE)
1	0.0000	0.0000	15.2500
2	1.0250	0.3750	1.3672
3	1.1875	0.4562	0.9777
4	1.2078	0.4888	0.9558
5	1.2048	0.5129	0.9442
6	1.1980	0.5354	0.9332
7	1.1906	0.5574	0.9226
8	1.1833	0.5790	0.9122

লক্ষ্য করলে দেখা যায় প্রথম Iteration-এর পর Loss অনেকটা কমে গিয়ে ধীরে ধীরে ছোট হতে থাকছে (15.2500 থেকে 0.9122-এ), যা প্রমাণ করে Model প্রতিটি Iteration-এ ভালোভাবে শিখছে এবং Convergence-এর দিকে এগোচ্ছে। বাস্তবে Algorithm এভাবেই শত-হাজার Iteration চালিয়ে যায় যতক্ষণ না Loss আর উল্লেখযোগ্যভাবে না কমে।

Final Learned Equation

৮টি Iteration শেষে Model-এর শেখা w ও b হলো:

$$w = 1.1761, \quad b = 0.6003$$

$$\hat{y} = 1.1761 \cdot X + 0.6003$$

উল্লেখযোগ্য বিষয় হলো — যদি Gradient Descent আরও বেশি Iteration চালানো হয়, তাহলে এই w ও b ধীরে ধীরে $w = 0.7$ এবং $b = 2.0$ -এর দিকে এগিয়ে যাবে, যা এই একই Dataset-এর সঠিক OLS সমাধান (দেখুন Chapter 3)।

নতুন Sample-এর জন্য Prediction

এখন এই শেখা Equation ব্যবহার করে একটি নতুন Sample, $X = 5$, Predict করা হলো:

$$\hat{y} = 1.1761 \times 5 + 0.6003 = 5.8804 + 0.6003 = 6.4807$$

সুতরাং $X = 5$ -এর জন্য Model-এর Prediction হলো $\hat{y} \approx 6.4807$ ।

Chapter 2 — Multiple Linear Regression

1. Introduction

মাল্টিপল লিনিয়ার রিগ্রেশন (Multiple Linear Regression) মূলত সিম্পল লিনিয়ার রিগ্রেশনেরই সম্প্রসারিত রূপ, যেখানে একাধিক ইনপুট ফিচার $x_1, x_2, \dots, x_m$ ব্যবহার করা হয়। বাস্তব জগতের প্রায় প্রতিটি সমস্যা একাধিক ফ্যাক্টরের উপর নির্ভর করে — যেমন বাড়ির দাম শুধু আয়তনের ওপর নয়, অবস্থান, বেডরুম সংখ্যা এবং বয়সের ওপরও নির্ভর করে।

2. Why This Algorithm Was Developed

Simple Linear Regression একটিমাত্র Feature ব্যবহার করে বলে এটি বাস্তব-জগতের জটিল সমস্যাগুলো ভালোভাবে ব্যাখ্যা করতে পারে না। একটি আউটপুট সাধারণত একাধিক কারণের সমন্বিত প্রভাবে নির্ধারিত হয়। এই সীমাবদ্ধতা দূর করতেই Multiple Linear Regression তৈরি হয়েছিল, যা একসাথে অনেকগুলো Independent Variable বিবেচনা করে একটি অধিক নির্ভুল ও বাস্তবসম্মত Model তৈরি করে।

3. Real-life Motivation

হাউজিং মার্কেটে বাড়ির দাম নির্ধারণ করতে আয়তন, অবস্থান, রুম সংখ্যা, বয়স — সবকিছু একসাথে প্রভাব ফেলে। কোনো প্রতিষ্ঠানের বিক্রয় পূর্বাভাস দিতে বিজ্ঞাপন বাজেট, খাতু, প্রতিযোগীর দাম — সব একসাথে হিসাব করতে হয়। Multiple Linear Regression এই ধরনের বহু-ফ্যাক্টর সমস্যার জন্য উপযুক্ত।

4. Intuition

Simple Linear Regression-এ আমরা 2D-তে একটি রেখা ফিট করি। Multiple Linear Regression-এ, একাধিক Feature থাকায় Model একটি Hyperplane (উচ্চ-মাত্রিক সমতল) ফিট করে, যা প্রতিটি Feature-এর জন্য একটি করে Weight শেখে — Feature যত গুরুত্বপূর্ণ, তার Weight তত বড়।

5. Working Principle

যখন একাধিক ফিচার থাকে, তখন প্রেডিকশন হয়:

$$\hat{y} = w_1X_1 + w_2X_2 + \dots + w_mX_m + b$$

এটিকে ভেক্টরাইজড ফর্মে লেখা হয়:

$$\hat{y} = XW + b$$

যেখানে x হলো ইনপুট ফিচার ম্যাট্রিক্স (সাইজ $n \times m$), W হলো ওয়েট ভেক্টর $[w_1, w_2, \dots, w_m]^T$, b হলো বায়াস (স্কেলার), এবং $\hat{y}$ হলো প্রেডিক্টেড আউটপুট।

6. Mathematical Backend

Mean Squared Error (MSE) কস্ট ফাংশন এখানে ম্যাট্রিক্স আকারে প্রকাশ করা হয়:

$$J(W,b) = (1/n) \sum_i (\hat{y}_i - y_i)^2 = (1/n) \|XW + b - y\|^2$$

7. Formula

$$\hat{y} = XW + b$$

8. Formula Derivation

ওয়েটের জন্য গ্রাডিয়েন্ট:

$$\partial J / \partial W = (2/n) X^T (XW + b - y)$$

বায়াসের জন্য গ্রাডিয়েন্ট:

$$\partial J / \partial b = (2/n) \sum_i (XW + b - y)$$

প্যারামিটার আপডেট:

$$W := W - \alpha \cdot \partial J / \partial W \quad b := b - \alpha \cdot \partial J / \partial b$$

9. Meaning of Every Symbol

Symbol	Meaning
X	ইনপুট ফিচার ম্যাট্রিক্স ($n \times m$)
W	ওয়েট ভেক্টর $[w_1, w_2, \dots, w_m]^T$
b	বায়াস (স্কেলার)
$\hat{y}$	প্রেডিক্টেড আউটপুট ভেক্টর
n	মোট Sample সংখ্যা
X^T	X ম্যাট্রিক্সের Transpose
α	Learning Rate

10. Step-by-Step Formula Explanation (Full Backend Workflow)

- Initialization: $W = 0, b = 0$
- Forward Pass: $\hat{y} = XW + b$
- Compute Cost: MSE দিয়ে error হিসাব
- Backward Pass: $X^T \cdot \text{error}$ ব্যবহার করে গ্রাডিয়েন্ট নির্ণয়
- Update Parameters: W এবং b আপডেট
- Convergence: লস মিনিমাম না হওয়া পর্যন্ত লুপ চলতে থাকে

11. Numerical Example

ধরা যাক একটি বাড়ির দাম নির্ভর করছে দুটি ফিচারের ওপর — আয়তন (x_1 , হাজার sq ft) ও বয়স (x_2 , বছর)। মডেল শেখার পর যদি $w_1 = 50$, $w_2 = -2$, $b = 10$ পাওয়া যায়, তাহলে 1,200 sq ft ও 5 বছর বয়সী একটি বাড়ির জন্য প্রেডিকশন হবে: $\hat{y} = 50(1.2) + (-2)(5) + 10 = 60 - 10 + 10 = 60$ (লক্ষ টাকায়)। লক্ষ করুন, প্রতিটি Feature-এর নিজস্ব Weight আছে, যা তার প্রভাবের দিক ও মাত্রা নির্দেশ করে।

12. Visualization

মাল্টিপল ফিচারের ক্ষেত্রে কস্ট ফাংশন একটি high-dimensional convex surface তৈরি করে। এটি দৃশ্যমানভাবে 2D বা 3D বাটি না হলেও গাণিতিকভাবে একটি মাত্র গ্লোবাল মিনিমাম থাকে।

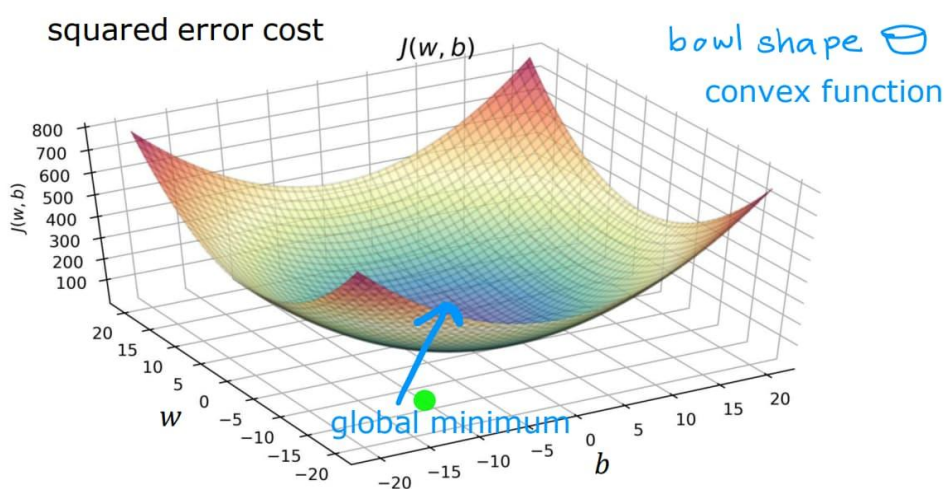


Figure 2.1 — Cost surface for multi-variable Linear Regression

13. Hyperparameters

Scikit-Learn-এ Multiple Linear Regression দুইভাবে বাস্তবায়িত হয় — LinearRegression (Closed-form OLS) এবং SGDRegressor (Gradient Descent ভিত্তিক)। LinearRegression-এ regularization বা learning-related কোনো parameter নেই, তাই টিউন করার মতো parameter খুবই কম।

LinearRegression — Hyperparameters

Parameter	Default	ব্যাখ্যা
fit_intercept	True	Model Bias Term (b) শিখবে কিনা তা নির্ধারণ করে। বাস্তবে প্রায় সব ক্ষেত্রেই True রাখা উচিত।
copy_X	True	Training-এর সময় Input Data-র একটি Copy তৈরি করে কাজ করবে কিনা।
n_jobs	None	কতগুলো CPU Core ব্যবহার হবে; -1 দিলে সব Core ব্যবহার হয়। Accuracy পরিবর্তন করে না, শুধু Speed বাড়ায়।

Parameter	Default	ব্যাখ্যা
positive	False	True করলে সব Coefficient (w) Non-negative-এ সীমাবদ্ধ থাকে।

SGDRegressor — Hyperparameters

SGDRegressor হলো Gradient Descent ভিত্তিক Model, যা Normal Equation ব্যবহার না করে ধাপে ধাপে Weight শেখে। এটি Large Dataset ও Online Learning-এর জন্য উপযোগী। এখানেই আসল hyperparameter tuning হয় এবং model performance-এর উপর সবচেয়ে বেশি প্রভাব ফেলে।

Parameter	Default	ব্যাখ্যা
loss	'squared_error'	Cost Function নির্ধারণ করে। Outlier থাকলে 'huber' ব্যবহার করা ভালো — এটি ছোট Error-এ Squared, বড় Error-এ Linear Loss ব্যবহার করে, ফলে Model বেশি Robust হয়।
penalty	'l2'	Regularization টাইপ নির্ধারণ করে: l2 (Ridge-স্টাইল, Weight ছোট করে), l1 (Lasso-স্টাইল, Weight শূন্য করে Feature Selection করে), অথবা elasticnet (উভয়ের সমন্বয়)।
alpha	0.0001	Regularization-এর Strength। বাড়ালে Model সহজ (কম Overfit) হয়, খুব বেশি বাড়ালে Underfitting হতে পারে।
l1_ratio	0.15	শুধু penalty='elasticnet' হলে কার্যকর। 0 হলে সম্পূর্ণ L2, 1 হলে সম্পূর্ণ L1।
learning_rate	'invscaling'	Training চলাকালে Learning Rate কীভাবে বদলাবে তা নির্ধারণ করে (constant/invscaling/adaptive)।
eta0	0.01	Initial Learning Rate।
max_iter	1000	সর্বোচ্চ কত Epoch Training চলবে।
tol	1e-3	Convergence Threshold — উন্নতি এর কম হলে Training থেমে যায়।
early_stopping	False	True করলে Validation Set ব্যবহার করে Overfitting হলে Training আগেই থামিয়ে দেয়।
validation_fraction	0.1	early_stopping=True হলে Validation-এর জন্য আলাদা করা Dataset-এর অংশ।
shuffle	True	প্রতিটি Epoch-এর আগে Data Shuffle হবে কিনা।
random_state	None	Random Operations Reproducible করে।
average	False	True করলে শেষ দিকের Weight-গুলোর Average নিয়ে Model আরও Stable করে।
warm_start	False	True করলে আগের Training-এর Weight থেকে পরবর্তী fit() শুরু হয়।

Parameter	Default	ব্যাখ্যা
epsilon	0.1	শুধু loss='huber'/'epsilon_insensitive'-এ কার্যকর — এই সীমার মধ্যে Error-কে Penalty দেওয়া হয় না।
n_iter_no_change	5	কত Epoch Loss না কমলে Early Stopping ধরা হবে।
power_t	0.5	শুধু learning_rate='invscaling'-এ কার্যকর — Learning Rate কত দ্রুত কমবে তা নির্ধারণ করে ($\eta = \eta_0/t^{\text{power_t}}$)।

COMMON MISTAKE

SGDRegressor ব্যবহারের আগে StandardScaler প্রয়োগ না করা — ফিচারের স্কেল বড় হলে গ্রাডিয়েন্ট হিসাবের সময় অত্যন্ত বড় মান তৈরি হতে পারে, যার ফলে ওয়েট হঠাৎ অনেক বেড়ে গিয়ে Model অস্থিতিশীল হয়ে যেতে পারে বা Cost Function NaN হয়ে যেতে পারে। LinearRegression-এর ক্ষেত্রে এই সমস্যা হয় না, কারণ এটি সরাসরি OLS সূত্র $(X^T X)^{-1} X^T y$ ব্যবহার করে একবারেই সমাধান বের করে — কোনো Iterative Learning প্রসেস নেই।

14. Scikit-learn Import

```
from sklearn.linear_model import LinearRegression, SGDRegressor
```

15. Python Example

From Scratch (No Scikit-Learn):

```
1 import numpy as np
2
3 X = np.array([
4     [1.0, 2100.0],
5     [2.0, 1500.0],
6     [3.0, 2800.0],
7     [4.0, 1200.0],
8     [5.0, 3200.0]
9 ])
10
11 y = np.array([45.0, 35.0, 60.0, 28.0, 70.0])
12
13 def predict(X, W, b):
14     return np.dot(X, W) + b
15
16 def compute_cost(X, y, W, b):
17     n = len(X)
18     return (1/n) * np.sum((predict(X, W, b) - y) ** 2)
19
20 def compute_gradient(X, y, W, b):
21     n = len(X)
22     error = predict(X, W, b) - y
23     dW = (2/n) * np.dot(X.T, error)
24     db = (2/n) * np.sum(error)
25     return dW, db
```

```

27 def gradient_descent(X, y, W, b, alpha, iters):
28     for i in range(iters):
29         dW, db = compute_gradient(X, y, W, b)
30
31         W = W - alpha * dW
32         b = b - alpha * db
33
34     return W, b
35
36 X_mean = np.mean(X, axis=0)
37 X_std = np.std(X, axis=0)
38 X_scaled = (X - X_mean) / X_std
39
40 W, b = gradient_descent(X_scaled, y, np.zeros(X.shape[1]), 0.0, 0.01, 50000)
41
42 print(W, b)

```

Scikit-Learn ব্যবহার করে:

```

1  from sklearn.linear_model import LinearRegression, SGDRegressor
2  from sklearn.preprocessing import StandardScaler
3
4  lr = LinearRegression()
5  lr.fit(X, y)
6
7  print(lr.coef_, lr.intercept_)
8
9  scaler = StandardScaler()
10 X_scaled = scaler.fit_transform(X)
11
12 sgd = SGDRegressor(max_iter=10000, eta0=0.01, random_state=42)
13 sgd.fit(X_scaled, y)
14
15 print(sgd.coef_)
16 print(sgd.predict(X_scaled[:1]))

```

16. Prediction Process

Training শেষে coef_ (W) এবং intercept_ (b) পাওয়া যায়। নতুন ডেটার জন্য প্রতিটি Feature-কে সংশ্লিষ্ট Weight দিয়ে গুণ করে যোগফলের সাথে Bias যোগ করলেই প্রেডিকশন পাওয়া যায় ($\hat{y} = XW + b$)। LinearRegression()-এর Attribute হিসেবে coef_, intercept_, rank_ (Design Matrix-এর Rank — Feature সংখ্যার চেয়ে কম হলে Multicollinearity নির্দেশ করে), singular_ (Numerical Stability বোঝায়), n_features_in_, এবং feature_names_in_ পাওয়া যায়। SGDRegressor()-এ অতিরিক্তভাবে n_iter_ (কতটি Epoch লেগেছে), t_ (মোট Weight Update সংখ্যা), এবং average=True হলে average_coef_ ও average_intercept_ পাওয়া যায়।

17. Advantages

- একাধিক ফ্যাক্টরের সম্মিলিত প্রভাব একসাথে মডেল করা যায়, ফলে বাস্তবসম্মত সমস্যায় বেশি নির্ভুল।
- প্রতিটি Feature-এর Coefficient দেখে তার আপেক্ষিক গুরুত্ব বোঝা যায় (Interpretability)।
- LinearRegression Closed-form solution দেয়, তাই ছোট-মাঝারি ডেটাসেটে দ্রুত ও Exact ফলাফল পাওয়া যায়।
- SGDRegressor বড় ডেটাসেট ও Online Learning পরিস্থিতিতে স্কেল করতে পারে।

18. Disadvantages

- Feature সংখ্যা বাড়লে Multicollinearity-এর ঝুঁকি বাড়ে, যা Coefficient-কে অস্থির করে তোলে।
- SGDRegressor Feature Scaling ছাড়া অস্থির হতে পারে — Cost Function NaN হয়ে যেতে পারে।
- এখনও একটি Linear মডেল, তাই জটিল Non-linear সম্পর্ক ধরতে পারে না।

19. Applications

- একাধিক ফ্যাক্টরের ভিত্তিতে বাড়ির দাম অনুমান (আয়তন, অবস্থান, বয়স, রুম সংখ্যা)।
- Business Forecasting — বিজ্ঞাপন বাজেট, খাতু, প্রতিযোগীর দাম একসাথে বিবেচনা করে বিক্রয় পূর্বাভাস।
- Large-scale Industrial ML সিস্টেমে Recommendation ও Real-time Prediction (SGDRegressor)।

20. Comparison with Previous Algorithm

দিক	Simple Linear Regression	Multiple Linear Regression
Feature সংখ্যা	একটি (X)	একাধিক ($X_1 \dots X_m$)
সমীকরণ	$\hat{y} = wX + b$	$\hat{y} = XW + b$
Geometric রূপ	2D রেখা	High-dimensional Hyperplane
Multicollinearity ঝুঁকি	নেই	থাকতে পারে
ব্যবহার-উপযোগিতা	একক-ফ্যাক্টর সমস্যা	বহু-ফ্যাক্টর বাস্তব সমস্যা

কখন LinearRegression এবং কখন SGDRegressor ব্যবহার করা উচিত, তার একটি সংক্ষিপ্ত গাইড:

ব্যবহার করুন	যখন
LinearRegression (OLS)	ডেটাসেট ছোট/মাঝারি, দ্রুত ও Exact সমাধান দরকার, ব্যাচ ট্রেনিং লাগবে না — যেমন house price prediction, academic projects
SGDRegressor	ডেটাসেট অনেক বড় (millions of rows), Online Learning দরকার, Memory সীমিত — যেমন recommendation system, real-time prediction

21. Common Mistakes

- SGDRegressor-এর আগে Feature Scaling না করা।
- সব Feature সরাসরি মডেলে ঢুকিয়ে দেওয়া, Multicollinearity যাচাই না করে।
- LinearRegression ও SGDRegressor-এর প্রয়োগক্ষেত্র গুলিয়ে ফেলা।

22. Interview Questions

Q: Multicollinearity কী এবং এটি কেন সমস্যা তৈরি করে?

A: যখন দুই বা ততোধিক Independent Feature নিজেদের মধ্যে অত্যন্ত সম্পর্কযুক্ত থাকে, তখন Model প্রতিটি Feature-এর পৃথক প্রভাব নির্ণয় করতে ব্যর্থ হয়, ফলে Coefficient অস্থির ও অবিশ্বাস্য হয়ে যায়।

Q: কেন Multiple Linear Regression-এ StandardScaler প্রায়ই ব্যবহৃত হয়?

A: LinearRegression (OLS)-এর জন্য বাধ্যতামূলক নয়, কিন্তু SGDRegressor-এর জন্য অত্যাৱশ্যক — না হলে গ্রাডিয়েন্ট হিসাব অস্থির হয়ে Training ভেঙে যেতে পারে।

23. Summary

Category	Key Points
Key Formula	$\hat{y} = XW + b$; $J(W,b) = (1/n) \ XW+b-y\ ^2$
Gradient	$\partial J / \partial W = (2/n) X^T (XW+b-y)$; $\partial J / \partial b = (2/n) \Sigma(\text{error})$
Two Implementations	LinearRegression (Closed-form OLS) vs SGDRegressor (Iterative)
Critical Practice	SGDRegressor-এর আগে অবশ্যই Feature Scaling করতে হবে
Exam Focus	Vector/Matrix ফর্মে Gradient ডেরাইভ করা
Interview Focus	Multicollinearity; কখন LinearRegression বনাম SGDRegressor

পরবর্তী অধ্যায়ে: এই Chapter-এ আমরা সংক্ষেপে দেখেছি যে LinearRegression সরাসরি একটি গাণিতিক সূত্র $(X^T X)^{-1} X^T y$ ব্যবহার করে সমাধান বের করে। Chapter 3-এ আমরা এই Ordinary Least Squares (OLS) পদ্ধতিটি গভীরভাবে বিশ্লেষণ করব — এর Derivation, প্রতিটি ধাপ, এবং হাতে-গণনাযোগ্য একটি সম্পূর্ণ Numerical Example দেখব।

24. Complete Mathematical Implementation (Full Manual Walkthrough)

এখানে Multiple Linear Regression-কে ২টি Feature-সহ একটি ছোট Dataset দিয়ে সম্পূর্ণ Manual Calculation-এ দেখানো হবে — কোনো Matrix লুকিয়ে না রেখে, প্রতিটি Weight আলাদাভাবে হিসাব করে।

Dataset

এই উদাহরণে ২টি Feature (X_1, X_2) এবং ৪টি Sample ব্যবহার করা হয়েছে:

Sample (i)	X1	X2	y (Actual)
1	1	3	9
2	2	1	6
3	3	5	15
4	4	2	10

Initialization

দুটি Weight এবং একটি Bias — সবগুলোকেই শূন্য দিয়ে শুরু করা হয়:

$$w1 = 0, w2 = 0, b = 0, \alpha = 0.02, n = 4$$

Forward Pass ও Formula

$$\hat{y}_i = w1 \cdot X1_i + w2 \cdot X2_i + b$$

$$J(w1, w2, b) = (1/n) \sum_i (\hat{y}_i - y_i)^2$$

প্রতিটি Weight-এর জন্য আলাদাভাবে Partial Derivative বের করা হয় — কারণ প্রতিটি Feature Cost-এ আলাদা প্রভাব ফেলে, তাই প্রতিটির Gradient আলাদাভাবে হিসাব করতে হয়:

$$\partial J / \partial w1 = (2/n) \sum_i (\hat{y}_i - y_i) \cdot X1_i \quad \partial J / \partial w2 = (2/n) \sum_i (\hat{y}_i - y_i) \cdot X2_i \quad \partial J / \partial b = (2/n) \sum_i (\hat{y}_i - y_i)$$

$$w1 := w1 - \alpha \cdot \partial J / \partial w1 \quad w2 := w2 - \alpha \cdot \partial J / \partial w2 \quad b := b - \alpha \cdot \partial J / \partial b$$

Iteration-by-Iteration Manual Calculation

Iteration 1 (শুরুর $w1 = 0.0000$, $w2 = 0.0000$, $b = 0.0000$)

বর্তমান $w1$, $w2$, b বসিয়ে প্রতিটি Sample-এর Prediction বের করা হলো:

$$\begin{aligned} \hat{y}_1 &= 0.0000 \times 1 + 0.0000 \times 3 + 0.0000 = 0.0000 & \hat{y}_2 &= 0.0000 \times 2 + 0.0000 \times 1 + 0.0000 = 0.0000 \\ \hat{y}_3 &= 0.0000 \times 3 + 0.0000 \times 5 + 0.0000 = 0.0000 & \hat{y}_4 &= 0.0000 \times 4 + 0.0000 \times 2 + 0.0000 = 0.0000 \end{aligned}$$

প্রতিটি Sample-এর Error ($\hat{y} - y$):

$$e1 = 0.0000 - 9 = -9.0000 \quad e2 = 0.0000 - 6 = -6.0000 \quad e3 = 0.0000 - 15 = -15.0000 \quad e4 = 0.0000 - 10 = -10.0000$$

$$J = (1/4) \cdot [(-9.0000)^2 + (-6.0000)^2 + (-15.0000)^2 + (-10.0000)^2] = 110.5000$$

এবার $w1$, $w2$ এবং b — প্রতিটির Gradient আলাদাভাবে বের করা হলো:

$$\partial J / \partial w_1 = (2/4) \cdot [(-9.0000 \times 1) + (-6.0000 \times 2) + (-15.0000 \times 3) + (-10.0000 \times 4)] = -53.0000$$

$$\partial J / \partial w_2 = (2/4) \cdot [(-9.0000 \times 3) + (-6.0000 \times 1) + (-15.0000 \times 5) + (-10.0000 \times 2)] = -64.0000$$

$$\partial J / \partial b = (2/4) \cdot [(-9.0000) + (-6.0000) + (-15.0000) + (-10.0000)] = -20.0000$$

Learning Rate $\alpha = 0.02$ দিয়ে প্রতিটি Parameter আলাদাভাবে Update করা হলো:

$$w_{1_new} = 0.0000 - 0.02 \times (-53.0000) = 1.0600$$

$$w_{2_new} = 0.0000 - 0.02 \times (-64.0000) = 1.2800$$

$$b_{_new} = 0.0000 - 0.02 \times (-20.0000) = 0.4000$$

Iteration 2 (শুরুর $w_1 = 1.0600$, $w_2 = 1.2800$, $b = 0.4000$)

বর্তমান w_1 , w_2 , b বসিয়ে প্রতিটি Sample-এর Prediction বের করা হলো:

$$\begin{aligned} \hat{y}_1 &= 1.0600 \times 1 + 1.2800 \times 3 + 0.4000 = 5.3000 & \hat{y}_2 &= 1.0600 \times 2 + 1.2800 \times 1 + 0.4000 = 3.8000 \\ \hat{y}_3 &= 1.0600 \times 3 + 1.2800 \times 5 + 0.4000 = 9.9800 & \hat{y}_4 &= 1.0600 \times 4 + 1.2800 \times 2 + 0.4000 = 7.2000 \end{aligned}$$

প্রতিটি Sample-এর Error ($\hat{y} - y$):

$$e_1 = 5.3000 - 9 = -3.7000 \quad e_2 = 3.8000 - 6 = -2.2000 \quad e_3 = 9.9800 - 15 = -5.0200 \quad e_4 = 7.2000 - 10 = -2.8000$$

$$J = (1/4) \cdot [(-3.7000)^2 + (-2.2000)^2 + (-5.0200)^2 + (-2.8000)^2] = 12.8926$$

এবার w_1 , w_2 এবং b — প্রতিটির Gradient আলাদাভাবে বের করা হলো:

$$\partial J / \partial w_1 = (2/4) \cdot [(-3.7000 \times 1) + (-2.2000 \times 2) + (-5.0200 \times 3) + (-2.8000 \times 4)] = -17.1800$$

$$\partial J / \partial w_2 = (2/4) \cdot [(-3.7000 \times 3) + (-2.2000 \times 1) + (-5.0200 \times 5) + (-2.8000 \times 2)] = -22.0000$$

$$\partial J / \partial b = (2/4) \cdot [(-3.7000) + (-2.2000) + (-5.0200) + (-2.8000)] = -6.8600$$

Learning Rate $\alpha = 0.02$ দিয়ে প্রতিটি Parameter আলাদাভাবে Update করা হলো:

$$w_{1_new} = 1.0600 - 0.02 \times (-17.1800) = 1.4036$$

$$w_{2_new} = 1.2800 - 0.02 \times (-22.0000) = 1.7200$$

$$b_{_new} = 0.4000 - 0.02 \times (-6.8600) = 0.5372$$

Iteration 3 (শুরুর $w_1 = 1.4036$, $w_2 = 1.7200$, $b = 0.5372$)

বর্তমান w_1, w_2, b বসিয়ে প্রতিটি Sample-এর Prediction বের করা হলো:

$$\begin{aligned}\hat{y}_1 &= 1.4036 \times 1 + 1.7200 \times 3 + 0.5372 = 7.1008 & \hat{y}_2 &= 1.4036 \times 2 + 1.7200 \times 1 + 0.5372 = 5.0644 \\ \hat{y}_3 &= 1.4036 \times 3 + 1.7200 \times 5 + 0.5372 = 13.3480 & \hat{y}_4 &= 1.4036 \times 4 + 1.7200 \times 2 + 0.5372 = 9.5916\end{aligned}$$

প্রতিটি Sample-এর Error ($\hat{y} - y$):

$$\begin{aligned}e_1 &= 7.1008 - 9 = -1.8992 & e_2 &= 5.0644 - 6 = -0.9356 & e_3 &= 13.3480 - 15 = -1.6520 \\ e_4 &= 9.5916 - 10 = -0.4084\end{aligned}$$

$$J = (1/4) \cdot [(-1.8992)^2 + (-0.9356)^2 + (-1.6520)^2 + (-0.4084)^2] = 1.8446$$

এবার w_1, w_2 এবং b — প্রতিটির Gradient আলাদাভাবে বের করা হলো:

$$\partial J / \partial w_1 = (2/4) \cdot [(-1.8992 \times 1) + (-0.9356 \times 2) + (-1.6520 \times 3) + (-0.4084 \times 4)] = -5.1800$$

$$\partial J / \partial w_2 = (2/4) \cdot [(-1.8992 \times 3) + (-0.9356 \times 1) + (-1.6520 \times 5) + (-0.4084 \times 2)] = -7.8550$$

$$\partial J / \partial b = (2/4) \cdot [(-1.8992) + (-0.9356) + (-1.6520) + (-0.4084)] = -2.4476$$

Learning Rate $\alpha = 0.02$ দিয়ে প্রতিটি Parameter আলাদাভাবে Update করা হলো:

$$w_{1_new} = 1.4036 - 0.02 \times (-5.1800) = 1.5072$$

$$w_{2_new} = 1.7200 - 0.02 \times (-7.8550) = 1.8771$$

$$b_{new} = 0.5372 - 0.02 \times (-2.4476) = 0.5862$$

Iteration 4 (শুরুর $w_1 = 1.5072, w_2 = 1.8771, b = 0.5862$)

বর্তমান w_1, w_2, b বসিয়ে প্রতিটি Sample-এর Prediction বের করা হলো:

$$\begin{aligned}\hat{y}_1 &= 1.5072 \times 1 + 1.8771 \times 3 + 0.5862 = 7.7247 & \hat{y}_2 &= 1.5072 \times 2 + 1.8771 \times 1 + 0.5862 = 5.4777 \\ \hat{y}_3 &= 1.5072 \times 3 + 1.8771 \times 5 + 0.5862 = 14.4933 & \hat{y}_4 &= 1.5072 \times 4 + 1.8771 \times 2 + 0.5862 = 10.3692\end{aligned}$$

প্রতিটি Sample-এর Error ($\hat{y} - y$):

$$\begin{aligned}e_1 &= 7.7247 - 9 = -1.2753 & e_2 &= 5.4777 - 6 = -0.5223 & e_3 &= 14.4933 - 15 = -0.5067 \\ e_4 &= 10.3692 - 10 = 0.3692\end{aligned}$$

$$J = (1/4) \cdot [(-1.2753)^2 + (-0.5223)^2 + (-0.5067)^2 + (0.3692)^2] = 0.5731$$

এবার w_1, w_2 এবং b — প্রতিটির Gradient আলাদাভাবে বের করা হলো:

$$\partial J / \partial w_1 = (2/4) \cdot [(-1.2753 \times 1) + (-0.5223 \times 2) + (-0.5067 \times 3) + (0.3692 \times 4)] = -1.1818$$

$$\partial J / \partial w_2 = (2/4) \cdot [(-1.2753 \times 3) + (-0.5223 \times 1) + (-0.5067 \times 5) + (0.3692 \times 2)] = -3.0719$$

$$\partial J / \partial b = (2/4) \cdot [(-1.2753) + (-0.5223) + (-0.5067) + (0.3692)] = -0.9676$$

Learning Rate $\alpha = 0.02$ দিয়ে প্রতিটি Parameter আলাদাভাবে Update করা হলো:

$$w_{1_new} = 1.5072 - 0.02 \times (-1.1818) = 1.5308$$

$$w_{2_new} = 1.8771 - 0.02 \times (-3.0719) = 1.9385$$

$$b_{new} = 0.5862 - 0.02 \times (-0.9676) = 0.6055$$

Iteration 5 (শুরুর $w_1 = 1.5308$, $w_2 = 1.9385$, $b = 0.6055$)

বর্তমান w_1 , w_2 , b বসিয়ে প্রতিটি Sample-এর Prediction বের করা হলো:

$$\begin{aligned} \hat{y}_1 &= 1.5308 \times 1 + 1.9385 \times 3 + 0.6055 = 7.9520 & \hat{y}_2 &= 1.5308 \times 2 + 1.9385 \times 1 + 0.6055 = 5.6057 \\ \hat{y}_3 &= 1.5308 \times 3 + 1.9385 \times 5 + 0.6055 = 14.8907 & \hat{y}_4 &= 1.5308 \times 4 + 1.9385 \times 2 + 0.6055 = 10.6059 \end{aligned}$$

প্রতিটি Sample-এর Error ($\hat{y} - y$):

$$e_1 = 7.9520 - 9 = -1.0480 \quad e_2 = 5.6057 - 6 = -0.3943 \quad e_3 = 14.8907 - 15 = -0.1093 \quad e_4 = 10.6059 - 10 = 0.6059$$

$$J = (1/4) \cdot [(-1.0480)^2 + (-0.3943)^2 + (-0.1093)^2 + (0.6059)^2] = 0.4082$$

এবার w_1 , w_2 এবং b — প্রতিটির Gradient আলাদাভাবে বের করা হলো:

$$\partial J / \partial w_1 = (2/4) \cdot [(-1.0480 \times 1) + (-0.3943 \times 2) + (-0.1093 \times 3) + (0.6059 \times 4)] = 0.1296$$

$$\partial J / \partial w_2 = (2/4) \cdot [(-1.0480 \times 3) + (-0.3943 \times 1) + (-0.1093 \times 5) + (0.6059 \times 2)] = -1.4365$$

$$\partial J / \partial b = (2/4) \cdot [(-1.0480) + (-0.3943) + (-0.1093) + (0.6059)] = -0.4728$$

Learning Rate $\alpha = 0.02$ দিয়ে প্রতিটি Parameter আলাদাভাবে Update করা হলো:

$$w_{1_new} = 1.5308 - 0.02 \times (0.1296) = 1.5282$$

$$w_{2_new} = 1.9385 - 0.02 \times (-1.4365) = 1.9673$$

$$b_{new} = 0.6055 - 0.02 \times (-0.4728) = 0.6150$$

Iteration 6 (শুরুর $w_1 = 1.5282$, $w_2 = 1.9673$, $b = 0.6150$)

বর্তমান w_1 , w_2 , b বসিয়ে প্রতিটি Sample-এর Prediction বের করা হলো:

$$\hat{y}_1 = 1.5282 \times 1 + 1.9673 \times 3 + 0.6150 = 8.0450 \quad \hat{y}_2 = 1.5282 \times 2 + 1.9673 \times 1 + 0.6150 = 5.6387 \quad \hat{y}_3 = 1.5282 \times 3 + 1.9673 \times 5 + 0.6150 = 15.0360 \quad \hat{y}_4 = 1.5282 \times 4 + 1.9673 \times 2 + 0.6150 = 10.6625$$

প্রতিটি Sample-এর Error ($\hat{y} - y$):

$$e_1 = 8.0450 - 9 = -0.9550 \quad e_2 = 5.6387 - 6 = -0.3613 \quad e_3 = 15.0360 - 15 = 0.0360 \quad e_4 = 10.6625 - 10 = 0.6625$$

$$J = (1/4) \cdot [(-0.9550)^2 + (-0.3613)^2 + (0.0360)^2 + (0.6625)^2] = 0.3707$$

এবার w_1 , w_2 এবং b — প্রতিটির Gradient আলাদাভাবে বের করা হলো:

$$\partial J / \partial w_1 = (2/4) \cdot [(-0.9550 \times 1) + (-0.3613 \times 2) + (0.0360 \times 3) + (0.6625 \times 4)] = 0.5402$$

$$\partial J / \partial w_2 = (2/4) \cdot [(-0.9550 \times 3) + (-0.3613 \times 1) + (0.0360 \times 5) + (0.6625 \times 2)] = -0.8606$$

$$\partial J / \partial b = (2/4) \cdot [(-0.9550) + (-0.3613) + (0.0360) + (0.6625)] = -0.3089$$

Learning Rate $\alpha = 0.02$ দিয়ে প্রতিটি Parameter আলাদাভাবে Update করা হলো:

$$w_{1_new} = 1.5282 - 0.02 \times (0.5402) = 1.5174$$

$$w_{2_new} = 1.9673 - 0.02 \times (-0.8606) = 1.9845$$

$$b_{new} = 0.6150 - 0.02 \times (-0.3089) = 0.6211$$

Iteration 7 (গুরুত্ব $w_1 = 1.5174$, $w_2 = 1.9845$, $b = 0.6211$)

বর্তমান w_1 , w_2 , b বসিয়ে প্রতিটি Sample-এর Prediction বের করা হলো:

$$\hat{y}_1 = 1.5174 \times 1 + 1.9845 \times 3 + 0.6211 = 8.0920 \quad \hat{y}_2 = 1.5174 \times 2 + 1.9845 \times 1 + 0.6211 = 5.6405 \quad \hat{y}_3 = 1.5174 \times 3 + 1.9845 \times 5 + 0.6211 = 15.0959 \quad \hat{y}_4 = 1.5174 \times 4 + 1.9845 \times 2 + 0.6211 = 10.6599$$

প্রতিটি Sample-এর Error ($\hat{y} - y$):

$$e_1 = 8.0920 - 9 = -0.9080 \quad e_2 = 5.6405 - 6 = -0.3595 \quad e_3 = 15.0959 - 15 = 0.0959 \quad e_4 = 10.6599 - 10 = 0.6599$$

$$J = (1/4) \cdot [(-0.9080)^2 + (-0.3595)^2 + (0.0959)^2 + (0.6599)^2] = 0.3496$$

এবার w_1 , w_2 এবং b — প্রতিটির Gradient আলাদাভাবে বের করা হলো:

$$\partial J / \partial w_1 = (2/4) \cdot [(-0.9080 \times 1) + (-0.3595 \times 2) + (0.0959 \times 3) + (0.6599 \times 4)] = 0.6500$$

$$\partial J / \partial w_2 = (2/4) \cdot [(-0.9080 \times 3) + (-0.3595 \times 1) + (0.0959 \times 5) + (0.6599 \times 2)] = -0.6422$$

$$\partial J / \partial b = (2/4) \cdot [(-0.9080) + (-0.3595) + (0.0959) + (0.6599)] = -0.2559$$

Learning Rate $\alpha = 0.02$ দিয়ে প্রতিটি Parameter আলাদাভাবে Update করা হলো:

$$w_{1_new} = 1.5174 - 0.02 \times (0.6500) = 1.5044$$

$$w_{2_new} = 1.9845 - 0.02 \times (-0.6422) = 1.9973$$

$$b_{_new} = 0.6211 - 0.02 \times (-0.2559) = 0.6263$$

Iteration 8 (শুরুর $w_1 = 1.5044$, $w_2 = 1.9973$, $b = 0.6263$)

বর্তমান w_1 , w_2 , b বসিয়ে প্রতিটি Sample-এর Prediction বের করা হলো:

$$\begin{aligned} \hat{y}_1 &= 1.5044 \times 1 + 1.9973 \times 3 + 0.6263 = 8.1227 & \hat{y}_2 &= 1.5044 \times 2 + 1.9973 \times 1 + 0.6263 = 5.6325 \\ \hat{y}_3 &= 1.5044 \times 3 + 1.9973 \times 5 + 0.6263 = 15.1262 & \hat{y}_4 &= 1.5044 \times 4 + 1.9973 \times 2 + 0.6263 = 10.6387 \end{aligned}$$

প্রতিটি Sample-এর Error ($\hat{y} - y$):

$$\begin{aligned} e_1 &= 8.1227 - 9 = -0.8773 & e_2 &= 5.6325 - 6 = -0.3675 & e_3 &= 15.1262 - 15 = 0.1262 \\ e_4 &= 10.6387 - 10 = 0.6387 \end{aligned}$$

$$J = (1/4) \cdot [(-0.8773)^2 + (-0.3675)^2 + (0.1262)^2 + (0.6387)^2] = 0.3322$$

এবার w_1 , w_2 এবং b — প্রতিটির Gradient আলাদাভাবে বের করা হলো:

$$\partial J / \partial w_1 = (2/4) \cdot [(-0.8773 \times 1) + (-0.3675 \times 2) + (0.1262 \times 3) + (0.6387 \times 4)] = 0.6604$$

$$\partial J / \partial w_2 = (2/4) \cdot [(-0.8773 \times 3) + (-0.3675 \times 1) + (0.1262 \times 5) + (0.6387 \times 2)] = -0.5456$$

$$\partial J / \partial b = (2/4) \cdot [(-0.8773) + (-0.3675) + (0.1262) + (0.6387)] = -0.2400$$

Learning Rate $\alpha = 0.02$ দিয়ে প্রতিটি Parameter আলাদাভাবে Update করা হলো:

$$w_{1_new} = 1.5044 - 0.02 \times (0.6604) = 1.4912$$

$$w_{2_new} = 1.9973 - 0.02 \times (-0.5456) = 2.0082$$

$$b_{_new} = 0.6263 - 0.02 \times (-0.2400) = 0.6311$$

Loss কীভাবে কমছে (Training Progress Table)

Iteration	w1	w2	Bias	Loss (MSE)
1	0.0000	0.0000	0.0000	110.5000

Iteration	w1	w2	Bias	Loss (MSE)
2	1.0600	1.2800	0.4000	12.8926
3	1.4036	1.7200	0.5372	1.8446
4	1.5072	1.8771	0.5862	0.5731
5	1.5308	1.9385	0.6055	0.4082
6	1.5282	1.9673	0.6150	0.3707
7	1.5174	1.9845	0.6211	0.3496
8	1.5044	1.9973	0.6263	0.3322

দেখা যাচ্ছে Loss 110.5000 থেকে কমে 0.3322-এ নেমে এসেছে, এবং w1, w2 দুটি ভিন্ন মান নিয়ে ভিন্নভাবে Converge করছে — কারণ X1 ও X2 আলাদা প্যাটার্নের Feature, তাই তাদের Weight-ও আলাদা হারে পরিবর্তিত হয়।

Final Learned Equation

$$w1 = 1.4912, w2 = 2.0082, b = 0.6311$$

$$\hat{y} = 1.4912 \cdot X1 + 2.0082 \cdot X2 + 0.6311$$

নতুন Sample-এর জন্য Prediction

নতুন Sample X1 = 5, X2 = 3-এর জন্য Prediction বের করা হলো:

$$\hat{y} = 1.4912 \times 5 + 2.0082 \times 3 + 0.6311 = 7.4562 + 6.0247 + 0.6311 = 14.1119$$

সুতরাং X1 = 5, X2 = 3-এর জন্য Model-এর Prediction হলো $\hat{y} \approx 14.1119$ ।

Chapter 3 — Ordinary Least Squares (OLS)

1. Introduction

পূর্ববর্তী দুই অধ্যায়ে আমরা Gradient Descent ব্যবহার করে ধাপে ধাপে Weight ও Bias শিখেছি। কিন্তু Linear Regression-এর জন্য একটি সরাসরি, একবারেই সমাধানকারী (Closed-form) গাণিতিক পদ্ধতিও আছে, যাকে বলা হয় Ordinary Least Squares (OLS)। এই অধ্যায়ে আমরা এই পদ্ধতিটি গভীরভাবে বিশ্লেষণ করব।

2. Why This Algorithm Was Developed

Gradient Descent একটি Iterative পদ্ধতি — এটি বহু ইটারেশনে ধীরে ধীরে Optimal সমাধানের দিকে এগোয়, এবং Learning Rate-এর মতো Hyperparameter সঠিকভাবে সেট করতে হয়। ঊনবিংশ শতকে গণিতবিদ Carl Friedrich Gauss ও Adrien-Marie Legendre লক্ষ্য করেন, Linear Regression-এর MSE Cost Function যেহেতু একটি Convex Quadratic ফাংশন, তাই Calculus ব্যবহার করে এর Minimum সরাসরি, একটি ধাপেই বের করা সম্ভব — কোনো Iteration বা Learning Rate ছাড়াই। এই আবিষ্কারই OLS-এর ভিত্তি।

3. Real-life Motivation

যখন ডেটাসেট ছোট বা মাঝারি আকারের (কয়েক হাজার সারি পর্যন্ত) এবং দ্রুত, নির্ভুল ও reproducible ফলাফল দরকার — যেমন Academic Research, ছোট Business Forecasting, বা কোনো One-time Analysis — তখন OLS Gradient Descent-এর চেয়ে বেশি উপযোগী, কারণ এতে Hyperparameter Tuning করার প্রয়োজন হয় না এবং ফলাফল সবসময় Exact।

4. Intuition

OLS-কে এভাবে কল্পনা করা যায় — মডেলটি বলছে, "আমি পুরো ডেটা একসাথে দেখে সবচেয়ে সঠিক রেখাটা একবারেই হিসাব করে ফেলছি," যেখানে Gradient Descent (বিশেষত SGD) বলে, "আমি অন্ধভাবে ধাপে ধাপে হাঁটছি, ভুল হলে দিক ঠিক করছি।" OLS তাই দ্রুত, স্থিতিশীল, এবং ছোট-মাঝারি ডেটাসেটের জন্য আদর্শ।

5. Working Principle

OLS হলো Linear Regression-এর একটি Closed-form Solution যেখানে মডেল কোনো Iteration ছাড়াই সরাসরি Best Parameters (W, b) বের করে ফেলে। OLS-এর লক্ষ্য:

$$\min_{W,b} J(W,b)$$

অর্থাৎ এমন W এবং b খুঁজে বের করা যাতে Error সবচেয়ে কম হয়। গণনা সহজ করার জন্য b -কে x -এর সাথে একটি Column হিসেবে যোগ করা হয় (Bias Trick):

$$X' = [X \ 1] \quad \hat{y} = X'\theta \quad \text{যেখানে } \theta = [W; b]$$

6. Mathematical Backend

OLS যেটা মিনিমাইজ করে তা হলো Mean Squared Error, যা মূলত prediction error-এর square-এর mean:

$$J(W,b) = (1/n) \|XW + b - y\|^2$$

Calculus ব্যবহার করে এই Cost Function-কে θ -এর সাপেক্ষে অন্তরীকরণ করে শূন্যের সমান বসালে ($\partial J/\partial \theta = 0$), সরাসরি Closed-form সমাধান পাওয়া যায় — এটিই OLS-এর মূল গাণিতিক ভিত্তি।

7. Formula

$$\theta = (X^T X)^{-1} X^T y$$

8. Formula Derivation — Step-by-step Mechanism

- Step 1 — Design Matrix তৈরি: $X \rightarrow X'$ (Bias-এর জন্য একটি অতিরিক্ত কলাম যোগ করা হয়)।
- Step 2 — Matrix Multiplication: $X^T X$ গণনা করা হয়।
- Step 3 — Inverse নেওয়া: $(X^T X)^{-1}$ বের করা হয়।
- Step 4 — Final Multiplication: $\theta = (X^T X)^{-1} X^T y$ — এটিই চূড়ান্ত সমাধান।

IMPORTANT NOTE

OLS তখনই কাজ করে যখন $(X^T X)$ Invertible (Non-singular) হয়। যদি Feature-গুলোর মধ্যে Perfect Multicollinearity থাকে, তাহলে এই Matrix Singular হয়ে যায় এবং Inverse বের করা সম্ভব হয় না — এক্ষেত্রে Ridge Regression (L2 Regularization) ব্যবহার করে সমস্যাটি সমাধান করা হয় (দেখুন Chapter 4)।

9. Meaning of Every Symbol

Symbol	Meaning
θ	সব Parameter একসাথে $[W; b]$
X'	Bias Column যুক্ত Design Matrix
X^T	X -এর Transpose
$(X^T X)^{-1}$	$X^T X$ ম্যাট্রিক্সের Inverse
y	Target ভেক্টর (আসল মান)

10. Step-by-Step Formula Explanation

উপরের চারটি ধাপ (Design Matrix $\rightarrow X^T X \rightarrow$ Inverse $\rightarrow$ Final Multiplication) অনুসরণ করলেই যেকোনো Dataset-এর জন্য Optimal θ একবারেই পাওয়া যায় — কোনো Learning Rate, কোনো Iteration Count, কোনো Convergence Check লাগে না।

11. Numerical Example

ধরা যাক আমাদের কাছে একটি ছোট dataset আছে:

X (Study Hours)	y (Marks)
1	2
2	4
3	6

আমরা চাই $\hat{y} = wX + b$ সম্পর্ক বের করতে। Design Matrix (Bias সহ):

$$X' = \begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{bmatrix} \quad y = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}^T$$

Step 1 — Transpose: $X'^T = \begin{bmatrix} 1 & 2 & 3 \\ 1 & 1 & 1 \end{bmatrix}$

Step 2 — $X'^T X'$ গুণ করে পাওয়া যায়: $\begin{bmatrix} 14 & 6 \\ 6 & 3 \end{bmatrix}$

Step 3 — Inverse: $(X'^T X')^{-1} = \begin{bmatrix} 1.5 & -3 \\ -3 & 7 \end{bmatrix}$

Step 4 — $X'^T y = \begin{bmatrix} 28 \\ 12 \end{bmatrix}^T$

চূড়ান্ত ফলাফল: $\theta = (X'^T X')^{-1} X'^T y = \begin{bmatrix} 2 \\ 0 \end{bmatrix}^T$, অর্থাৎ $w = 2$, $b = 0$ ।

$$\text{Final Model: } \hat{y} = 2X + 0$$

EXAM TIP

এই ছোট, হাতে-গণনাযোগ্য Example ($w=2$, $b=0$) পরীক্ষায় OLS Derivation বোঝাতে প্রায়ই আসে — Design Matrix তৈরি থেকে Final θ পর্যন্ত প্রতিটি ধাপ আলাদাভাবে দেখাতে পারলে পূর্ণ নম্বর পাওয়া যায়।

12. Visualization

$$X \rightarrow [X'^T X']^{-1} \rightarrow \times X'^T y \rightarrow \theta (w, b) \rightarrow \hat{y} = X\theta$$

উপরের ডায়াগ্রামটি দেখায় কীভাবে Raw Feature Matrix থেকে সরাসরি একটি একক Matrix গণনা-শৃঙ্খলের মাধ্যমে চূড়ান্ত Parameter পাওয়া যায় — কোনো লুপ ছাড়াই।

13. Hyperparameters

যেহেতু OLS একটি Closed-form গাণিতিক সমাধান, তাই এতে Learning Rate বা Iteration Count-এর মতো কোনো Training Hyperparameter নেই। Scikit-Learn-এর LinearRegression ক্লাস (যা OLS ব্যবহার করে) কেবল Configuration-সংক্রান্ত কয়েকটি Parameter সরবরাহ করে (দেখুন Chapter 2, Section 13): `fit_intercept`, `copy_X`, `n_jobs`, এবং `positive`।

14. Scikit-learn Import


```
from sklearn.linear_model import LinearRegression
import numpy as np
```

15. Python Example

উপরের ম্যানুয়াল উদাহরণটি NumPy দিয়ে সরাসরি Matrix ফর্মুলা ব্যবহার করে যাচাই করা যায়:

```
X = np.array([[1,1],[2,1],[3,1]])
y = np.array([2,4,6])

theta = np.linalg.inv(X.T @ X) @ X.T @ y
print(theta) # [2. 0.]

# Scikit-Learn equivalent
model = LinearRegression()
model.fit([[1],[2],[3]], [2,4,6])
print(model.coef_, model.intercept_) # [2.] 0.0
```

16. Prediction Process

একবার θ (অর্থাৎ w ও b) গণনা হয়ে গেলে, নতুন যেকোনো x -এর জন্য শুধুমাত্র $\hat{y} = x'\theta$ সূত্রে বসিয়ে সরাসরি প্রেডিকশন পাওয়া যায় — কোনো পুনরায় Training দরকার হয় না।

17. Advantages

- Exact, Closed-form সমাধান — কোনো Approximation নেই।
- কোনো Learning Rate বা Iteration Count টিউন করার প্রয়োজন নেই।
- ছোট-মাঝারি ডেটাসেটে অত্যন্ত দ্রুত।
- Reproducible — একই ডেটায় সবসময় একই ফলাফল দেয়।

18. Disadvantages

- $X^T X$ Invertible না হলে (Perfect Multicollinearity) কাজ করে না।
- বড় Feature সংখ্যায় Matrix Inversion কম্পিউটেশনালি ব্যয়বহুল ($O(m^3)$ জটিলতা)।
- খুব বড় Dataset-এ (millions of rows) Memory-তে পুরো X ম্যাট্রিক্স রাখা কঠিন হয়ে পড়ে।

19. Applications

- House Price Prediction-এর মতো ছোট-মাঝারি Business Forecasting।
- Academic ও Research Projects, যেখানে Exact Coefficient প্রয়োজন।
- Statistical Analysis যেখানে Confidence Interval ও Hypothesis Testing দরকার।

20. Comparison with Previous Algorithm

দিক	OLS (Closed-form)	SGD (Iterative)
পদ্ধতি	সরাসরি সূত্র $(X^T X)^{-1} X^T y$	ধাপে ধাপে Gradient Update
Hyperparameter	প্রয়োজন নেই	Learning Rate, Iteration ইত্যাদি
ডেটাসেট সাইজ	ছোট-মাঝারি	বড়-অতিবৃহৎ
Feature Scaling	সাধারণত প্রয়োজন নেই	প্রায় বাধ্যতামূলক
Memory ব্যবহার	বেশি (পুরো Matrix)	কম (Batch/Online)

21. Common Mistakes

- Multicollinearity যাচাই না করে সরাসরি OLS প্রয়োগ করা, ফলে Matrix Singular হয়ে যাওয়া।
- বিশাল Dataset-এ OLS ব্যবহার করে Memory Overflow-এর সম্মুখীন হওয়া — এক্ষেত্রে SGDRegressor উপযুক্ত।
- OLS এবং Gradient Descent-কে একই জিনিস মনে করা।

22. Interview Questions

Q: OLS এবং Gradient Descent-এর মধ্যে পার্থক্য কী?

A: OLS একটি Closed-form, একবারেই সমাধানকারী পদ্ধতি; Gradient Descent ধাপে ধাপে Iterative ভাবে Optimal মান খুঁজে বের করে।

Q: $X^T X$ Singular হলে কী করবেন?

A: এর অর্থ Feature-গুলোর মধ্যে Perfect Multicollinearity আছে। এক্ষেত্রে Ridge Regression ব্যবহার করা হয়, যা Diagonal-এ একটি ছোট মান (λ) যোগ করে Matrix-কে Invertible করে তোলে — দেখুন Chapter 4।

23. Summary

Category	Key Points
Key Formula	$\theta = (X^T X)^{-1} X^T y$
Nature	Closed-form, non-iterative, exact solution

Category	Key Points
Requirement	$X^T X$ must be invertible (no perfect multicollinearity)
Best For	Small–medium datasets requiring exact, fast solutions
Exam Focus	Full step-by-step derivation with a small numeric dataset
Interview Focus	OLS vs Gradient Descent; what happens when $X^T X$ is singular

পরবর্তী অধ্যায়ে: OLS নিখুঁত কাজ করে যতক্ষণ না Feature-গুলোর মধ্যে Multicollinearity থাকে বা Model Overfit করে। Chapter 4-এ আমরা দেখব কীভাবে Ridge Regression একটি Penalty Term যোগ করে এই সমস্যা সমাধান করে এবং $X^T X$ -কে সবসময় Invertible রাখে।

24. Complete Mathematical Implementation (Full Manual Walkthrough)

OLS Gradient Descent ছাড়াই সরাসরি Matrix Algebra দিয়ে w ও b বের করে, তাই এখানে পুরো Matrix পথটি — Design Matrix থেকে শুরু করে Final θ পর্যন্ত — ধাপে ধাপে Manual-ভাবে দেখানো হবে।

Dataset

এখানে Chapter 1-এর মতোই ছোট একটি Dataset ব্যবহার করা হয়েছে, যাতে Manual Matrix গণনা সহজ হয়:

Sample (i)	X	y
1	1	2
2	2	4
3	3	5
4	4	4

Step 1 — Design Matrix (X)

Bias (Intercept)-কে Matrix-এর ভেতরেই ধরার জন্য x -এর সামনে একটি অতিরিক্ত কলাম (সব মান 1) বসানো হয়:

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix} \quad (\text{কলাম ১} = \text{Bias-এর জন্য, কলাম ২} = \text{Feature } X)$$

Step 2 — Transpose (X^T)

X^T পেতে হলে Matrix-এর সারি ও কলাম অদলবদল করতে হয়:

$$X^T = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \end{bmatrix}$$

Step 3 — $X^T X$ (Matrix Multiplication)

X^T ও X -কে গুণ করে একটি 2×2 Matrix পাওয়া যায়। প্রতিটি Entry বের করতে সংশ্লিষ্ট সারি ও কলামের Element-গুলো একে একে গুণ করে যোগ করতে হয়:

$$\begin{aligned}X^T X[0][0] &= (1 \times 1) + (1 \times 1) + (1 \times 1) + (1 \times 1) = 4 \\X^T X[0][1] &= (1 \times 1) + (1 \times 2) + (1 \times 3) + (1 \times 4) = 10 \\X^T X[1][0] &= (1 \times 1) + (2 \times 1) + (3 \times 1) + (4 \times 1) = 10 \\X^T X[1][1] &= (1 \times 1) + (2 \times 2) + (3 \times 3) + (4 \times 4) = 1 + 4 + 9 + 16 = 30 \\X^T X &= [[4, 10], [10, 30]]\end{aligned}$$

Step 4 — Determinant

2×2 Matrix $[[a, b], [c, d]]$ -এর Determinant হয় $(a \times d - b \times c)$:

$$\det(X^T X) = (4 \times 30) - (10 \times 10) = 120 - 100 = 20.0$$

Step 5 — Inverse Matrix

2×2 Matrix $[[a, b], [c, d]]$ -এর Inverse হয় $(1/\det) \times [[d, -b], [-c, a]]$ — কারণ $M \cdot M^{-1} = I$ শর্তটি পূরণ করতেই এই Formula আসে:

$$(X^T X)^{-1} = (1/20.0) \times [[30, -10], [-10, 4]] = [[1.5000, -0.5000], [-0.5000, 0.2000]]$$

Step 6 — $X^T y$

এবার X^T -কে y ভেক্টরের সাথে গুণ করা হলো:

$$\begin{aligned}X^T y[0] &= (1 \times 2) + (1 \times 4) + (1 \times 5) + (1 \times 4) = 15.0 \\X^T y[1] &= (1 \times 2) + (2 \times 4) + (3 \times 5) + (4 \times 4) = 2 + 8 + 15 + 16 = 41.0 \\X^T y &= [15.0, 41.0]\end{aligned}$$

Step 7 — Final Multiplication $\rightarrow \theta$

$\theta = (X^T X)^{-1} \cdot X^T y$ — এই Matrix-Vector গুণটিই আমাদের চূড়ান্ত উত্তর দেয়:

$$\begin{aligned}\theta[0] (b) &= (1.5000 \times 15.0) + (-0.5000 \times 41.0) = 22.5000 + -20.5000 = 2.0000 \\ \theta[1] (w) &= (-0.5000 \times 15.0) + (0.2000 \times 41.0) = -7.5000 + 8.2000 = 0.7000\end{aligned}$$

Final Equation

$$\begin{aligned}b &= 2.0000, \quad w = 0.7000 \\ \hat{y} &= 0.7000 \cdot X + 2.0000\end{aligned}$$

লক্ষ্যণীয় — Chapter 1-এ একই Dataset-এ Gradient Descent যে w ও b -এর দিকে ধীরে ধীরে এগোচ্ছিল, এই Matrix পদ্ধতি ঠিক সেই মানই এক ধাপেই (কোনো Iteration ছাড়াই) সরাসরি বের করে দিলো — এটাই OLS-এর মূল সুবিধা।

নতুন Sample-এর জন্য Prediction

নতুন Sample $X = 5$ -এর জন্য Prediction:

$$\hat{y} = 0.7000 \times 5 + 2.0000 = 3.5000 + 2.0000 = 5.5000$$

সুতরাং $X = 5$ -এর জন্য OLS Model-এর Prediction হলো $\hat{y} = 5.5000$ ।

Chapter 4 — Ridge Regression

1. Introduction

Ridge Regression হলো Linear Regression-এর একটি উন্নত সংস্করণ যেখানে L2 Regularization ব্যবহার করা হয়। এটি Loss Function-এর সাথে প্রতিটি Weight-এর Square যোগ করে, যাতে বড় Weight-এর জন্য বেশি Penalty হয় এবং Model সব Weight ছোট করার চেষ্টা করে।

2. Why This Algorithm Was Developed

Linear Regression-এ Model এমনভাবে Coefficient শেখে যাতে Prediction Error (MSE) সর্বনিম্ন হয়। কিন্তু অনেক সময় Model কিছু Feature-এর জন্য অত্যন্ত বড় Weight শিখে ফেলে, যার ফলে Overfitting এবং Multicollinearity-এর মতো সমস্যা দেখা দেয়। এছাড়া Chapter 3-এ আমরা দেখেছি, Multicollinearity থাকলে $X^T X$ Matrix Singular হয়ে যায় এবং OLS সমাধান বের করতে ব্যর্থ হয়। এই দুটি সমস্যা সমাধানের জন্যই Ridge Regression (Arthur Hoerl ও Robert Kennard, ১৯৭০) তৈরি করা হয়েছিল।

3. Real-life Motivation

যেসব Dataset-এ সব Feature গুরুত্বপূর্ণ, কিন্তু Feature-গুলোর মধ্যে Multicollinearity রয়েছে অথবা Overfitting কমাতে হয়, সেখানে Ridge সবচেয়ে উপযোগী — যেমন জিনতত্ত্ব গবেষণায় (Genomics) হাজারো Correlated Gene-Feature নিয়ে কাজ করা, অথবা আর্থিক মডেলে একাধিক পরস্পর-সম্পর্কিত Economic Indicator ব্যবহার করা।

4. Intuition

Ridge-কে একটি "ব্রেক" হিসেবে কল্পনা করা যায় যা Weight-কে অতিরিক্ত বড় হতে বাধা দেয়। OLS শুধু Error কমাতে চায়, তাই কখনো কখনো Noise ফিট করতে গিয়ে Weight অস্বাভাবিক বড় হয়ে যায়। Ridge Loss Function-এ w^2 যোগ করে Model-কে বলে দেয় — "Error কমাও, কিন্তু Weight-কেও সংযত রাখো।"

5. Working Principle

L2 Regularization-এ Loss Function-এর সাথে সকল Weight-এর Square (w^2) যোগ করা হয়। এর ফলে বড় Weight-এর জন্য বেশি Penalty হয় এবং Model সব Weight ছোট করার চেষ্টা করে। তবে L2 সাধারণত কোনো Weight-কে সম্পূর্ণ 0 করে না — অর্থাৎ সব Feature Model-এ থেকে যায়, শুধু তাদের গুরুত্ব (Weight) কমে যায়।

6. Mathematical Backend

Ridge-এর Cost Function মূল MSE-এর সাথে একটি Penalty Term যোগ করে গঠিত:

$$J(W,b) = \text{MSE} + \lambda \sum_i w_i^2$$

7. Formula

$$\text{Loss} = \text{MSE} + \lambda \sum_i w_i^2$$

8. Formula Derivation

OLS-এর মতো এখানেও θ -এর সাপেক্ষে Cost Function-কে অন্তরীকরণ করে শূন্যের সমান বসানো হয়। ফলাফল হিসেবে একটি Closed-form সমাধান পাওয়া যায়, যা OLS সমাধানের অনুরূপ কিন্তু Diagonal-এ একটি λI Term যোগ হয়:

$$\theta_{\text{ridge}} = (X^T X + \lambda I)^{-1} X^T y$$

IMPORTANT NOTE

লক্ষ করুন, $X^T X$ -এর সাথে λI (λ গুণিতক Identity Matrix) যোগ করা হয়েছে। এই কারণেই Ridge সবসময় Invertible থাকে, এমনকি Multicollinearity থাকলেও — কারণ λI যোগ করার ফলে Matrix আর কখনো Singular হয় না। এটিই Chapter 3-এ উল্লেখিত OLS-এর সীমাবদ্ধতার সরাসরি সমাধান।

9. Meaning of Every Symbol

Symbol	Meaning
MSE	Mean Squared Error (মূল Prediction Error)
λ (Lambda / alpha)	Regularization Parameter — Penalty কতটা শক্তিশালী হবে তা নিয়ন্ত্রণ করে
w_i	প্রতিটি Feature-এর Coefficient/Weight
I	Identity Matrix

10. Step-by-Step Formula Explanation

- মূল MSE গণনা করা হয়, যেমন OLS-এ হতো।
- প্রতিটি Weight-এর Square যোগ করে একটি Penalty Term তৈরি করা হয়।
- λ দিয়ে এই Penalty-র শক্তি নিয়ন্ত্রণ করা হয় — $\lambda = 0$ হলে এটি সাধারণ Linear Regression-এর সমান হয়ে যায়।
- Closed-form সমাধান $(X^T X + \lambda I)^{-1} X^T y$ ব্যবহার করে সরাসরি Weight বের করা হয় (অথবা Gradient Descent দিয়েও সমাধান করা যায়)।

11. Numerical Example

ধরো একটি Model-এর OLS Training-এর পরে পাওয়া Weight ছিল:

Feature	OLS Weight ($\lambda=0$)	Ridge Weight (λ বড়)
Age	25	15
Salary	40	22
Balance	18	11
Gender	8	4
City	5	3

লক্ষ্য করো, Ridge প্রয়োগ করার পর সব Weight ছোট হয়েছে, কিন্তু কোনো Weight শূন্য হয়নি — Model সব Feature ব্যবহার করছে, কিন্তু কোনো Feature-কে অতিরিক্ত গুরুত্ব দিচ্ছে না।

12. Visualization

OLS Weight vector (large, unconstrained) $\rightarrow$ shrink toward 0 by $\lambda \|w\|^2 \rightarrow$ Ridge Weight vector (small, none exactly zero)

13. Hyperparameters

Parameter	Default	ব্যাখ্যা
alpha	1.0	Regularization Strength (λ)। বাড়ালে Weight আরও ছোট হয় (Underfitting-এর ঝুঁকি); কমালে OLS-এর কাছাকাছি আচরণ করে (Overfitting-এর ঝুঁকি)।
fit_intercept	True	Bias Term (b) শেখা হবে কিনা।
solver	'auto'	Closed-form সমাধানের জন্য Numerical Algorithm বেছে নেয় (svd, cholesky, lsqr, sag ইত্যাদি) — ডেটার আকার অনুযায়ী auto সবচেয়ে উপযুক্তটি বেছে নেয়।
max_iter	None	Iterative solver (যেমন sag/saga) ব্যবহার করলে সর্বোচ্চ Iteration সংখ্যা।
tol	1e-4	Iterative Solver-এর Convergence Threshold।
random_state	None	Stochastic Solver (sag/saga) ব্যবহারের সময় Reproducibility নিশ্চিত করে।

PRACTICAL TIP

Ridge ব্যবহারের আগে Feature Scaling (StandardScaler) করা অত্যন্ত গুরুত্বপূর্ণ, কারণ Penalty Term ($\lambda \sum w^2$) সরাসরি Weight-এর মানের ওপর নির্ভর করে — স্কেল না করলে বড় Range-এর Feature অন্যায়ভাবে বেশি Penalty পাবে বা কম Penalty পাবে।

14. Scikit-learn Import

```
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
```


15. Python Example

```
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

model = make_pipeline(StandardScaler(), Ridge(alpha=1.0))

model.fit(X_train, y_train)

predictions = model.predict(X_test)

print(model[-1].coef_, model[-1].intercept_)
```

16. Prediction Process

Training শেষে coef_ এবং intercept_ পাওয়া যায়, ঠিক LinearRegression-এর মতোই। প্রেডিকশন একই সূত্রে হয়: $\hat{y} = XW + b$ । পার্থক্য শুধু Training-এর সময়ে — কীভাবে W নির্ধারিত হয়েছে তাতে (Penalty Term-সহ)।

17. Advantages

- Multicollinearity সমস্যা সমাধান করে — $X^T X + \lambda I$ সবসময় Invertible।
- Overfitting কমায়, বিশেষত Feature সংখ্যা বেশি হলে।
- সব Feature Model-এ রাখে, তাই কোনো তথ্য সম্পূর্ণ হারায় না।
- Closed-form সমাধান বিদ্যমান, তাই দ্রুত।

18. Disadvantages

- Feature Selection করে না — সব Feature থেকেই যায়, এমনকি অপ্রয়োজনীয়গুলোও।
- সঠিক λ (alpha) খুঁজে বের করতে Cross-Validation প্রয়োজন।
- Feature Scaling ছাড়া Penalty Term অসামঞ্জস্যপূর্ণভাবে কাজ করে।

19. Applications

- Genomics ও Bioinformatics — হাজারো Correlated Gene-Feature নিয়ে কাজ করার সময়।

- আর্থিক Risk Modeling, যেখানে বহু Economic Indicator পরস্পর-সম্পর্কিত।
- যেকোনো High-dimensional Dataset যেখানে সব Feature গুরুত্বপূর্ণ কিন্তু Overfitting এড়াতে হবে।

20. Comparison with Previous Algorithm

দিক	OLS	Ridge Regression
Regularization	নেই	L2 (Σw^2)
Multicollinearity	সমস্যা তৈরি করে (Singular Matrix)	সমাধান করে ($X^T X + \lambda I$ সবসময় Invertible)
Feature Selection	না	না (Weight শূন্য হয় না)
Overfitting নিয়ন্ত্রণ	নেই	আছে (λ দিয়ে)

21. Common Mistakes

- Feature Scaling ছাড়া Ridge ব্যবহার করা।
- খুব বড় alpha সেট করে Model-কে Underfit করে ফেলা।
- Ridge Feature Selection করবে ভেবে ভুল প্রত্যাশা রাখা — এটি শুধু Lasso করে (দেখুন Chapter 5)।

22. Interview Questions

Q: Ridge কেন কোনো Weight-কে ঠিক শূন্য করে না?

A: L2 Penalty (w^2)-এর Derivative w -এর সমানুপাতিক, যা w শূন্যের কাছাকাছি গেলে খুব ছোট হয়ে যায় — ফলে Optimization Process Weight-কে শূন্যের কাছাকাছি আনে কিন্তু ঠিক শূন্যে পৌঁছাতে বাধ্য করে না। L1 Penalty ($|w|$)-এর ক্ষেত্রে এমনটা হয় না, তাই Lasso Weight-কে সম্পূর্ণ শূন্য করতে পারে।

Q: alpha = 0 করলে Ridge কী হয়ে যায়?

A: Penalty Term সম্পূর্ণ অদৃশ্য হয়ে যায়, ফলে Ridge ঠিক সাধারণ OLS Linear Regression-এর সমান হয়ে যায়।

23. Summary

Category	Key Points
Key Formula	$Loss = MSE + \lambda \Sigma w_i^2$
Closed-form	$\theta = (X^T X + \lambda I)^{-1} X^T y$
Effect on Weights	সব Weight ছোট হয়, কোনোটা শূন্য হয় না
Solves	Multicollinearity ও Overfitting
Exam Focus	λ যোগ করলে কেন Matrix সবসময় Invertible থাকে তা ব্যাখ্যা করা

Category	Key Points
Interview Focus	Ridge বনাম OLS বনাম Lasso; alpha-এর ভূমিকা

পরবর্তী অধ্যায়ে: Ridge সব Feature-কে ছোট করে রাখে কিন্তু বাদ দেয় না। যদি Dataset-এ অনেক অপয়োজনীয় Feature থাকে এবং আমরা চাই Model নিজে থেকেই সেগুলো বাদ দিক, তাহলে Chapter 5-এ আমরা শিখব Lasso Regression, যা L1 Regularization ব্যবহার করে স্বয়ংক্রিয় Feature Selection করে।

24. Complete Mathematical Implementation (Full Manual Walkthrough)

Ridge Regression, OLS-এরই একটি সম্প্রসারণ — এখানে একই Dataset ও একই $X^T X / X^T y$ ব্যবহার করে দেখানো হবে কীভাবে একটি Penalty Term যোগ করলে Weight-এর মান পরিবর্তিত হয়।

OLS-এর Dataset ব্যবহার

Chapter 3 (OLS)-এর একই Dataset ব্যবহার করা হচ্ছে, এবং সেখান থেকে ইতিমধ্যে বের করা $X^T X$ ও $X^T y$ সরাসরি এখানে পুনরায় ব্যবহার করা হবে:

Sample (i)	X	y
1	1	2
2	2	4
3	3	5
4	4	4

$$X^T X = \begin{bmatrix} 4 & 10 \\ 10 & 30 \end{bmatrix} \quad X^T y = \begin{bmatrix} 15 \\ 41 \end{bmatrix}$$

Penalty যোগ করা

Ridge Regression, Cost Function-এ Weight-এর বর্গের একটি Penalty যোগ করে, যাতে খুব বড় Weight-কে নিরুৎসাহিত করা যায় এবং Overfitting কমে:

$$J_{\text{ridge}}(w, b) = (1/n) \sum_i (\hat{y}_i - y_i)^2 + \lambda \cdot w^2$$

এখানে শুধু w -কে Penalize করা হচ্ছে, b (Bias)-কে নয় — কারণ Bias শুধু Baseline মান নির্দেশ করে, এটি বড় হলে Overfitting হয় না, তাই একে Regularize করার প্রয়োজন নেই।

λ নির্বাচন

এই উদাহরণের জন্য একটি মাঝারি মানের Penalty বাছাই করা হলো:

$$\lambda = 2$$

$X^T X + \lambda R$ (R = শুধু w -এর অবস্থানে 1, বাকি সব 0)

যেহেতু Bias-কে Regularize করা হচ্ছে না, তাই Identity Matrix-এর বদলে একটি Matrix R ব্যবহার করা হলো যেখানে শুধু w-এর সাথে সম্পর্কিত Entry-তে 1 আছে:

$$R = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$$

$$X^T X + \lambda R = \begin{bmatrix} 4 & 10 \\ 10 & 30 + 2 \times 1 \end{bmatrix} = \begin{bmatrix} 4 & 10 \\ 10 & 32 \end{bmatrix}$$

Inverse বের করা

$$\det = (4 \times 32) - (10 \times 10) = 128 - 100 = 28.0$$

$$(X^T X + \lambda R)^{-1} = (1/28.0) \times \begin{bmatrix} 32 & -10 \\ -10 & 4 \end{bmatrix} = \begin{bmatrix} 1.1429 & -0.3571 \\ -0.3571 & 0.1429 \end{bmatrix}$$

Final Weight বের করা

$$\theta[0] (b) = (1.1429 \times 15) + (-0.3571 \times 41) = 17.1429 + -14.6429 = 2.5000$$

$$\theta[1] (w) = (-0.3571 \times 15) + (0.1429 \times 41) = -5.3571 + 5.8571 = 0.5000$$

$$b = 2.5000, \quad w = 0.5000$$

OLS-এর সাথে Comparison

Model	b (Bias)	w (Weight)
OLS ($\lambda=0$)	2.0000	0.7000
Ridge ($\lambda=2$)	2.5000	0.5000

OLS-এ $w = 0.7000$ ছিল, Ridge-এ তা কমে $w = 0.5000$ -এ নেমে এসেছে। অর্থাৎ Penalty যোগ করার ফলে w-এর মান শূন্যের দিকে সংকুচিত (Shrink) হয়েছে, যদিও b কিছুটা বেড়েছে যাতে সামগ্রিক Prediction তুলনামূলকভাবে ঠিক থাকে।

Weight কেন ছোট হলো – Mathematical ব্যাখ্যা

$X^T X$ -এর w-সম্পর্কিত Entry ছিল 30, Ridge-এ তার সাথে $\lambda = 2$ যোগ হয়ে তা 32 হয়েছে – অর্থাৎ Denominator বড় হয়েছে অথচ Numerator ($X^T y$) অপরিবর্তিত। যেহেতু $\theta = (X^T X + \lambda R)^{-1} \cdot X^T y$ -তে বড় Denominator মানে ছোট Inverse Value, তাই চূড়ান্ত w-ও ছোট হয়ে গেছে। এভাবেই λ বাড়ালে Ridge-এর Weight ধারাবাহিকভাবে আরও ছোট হতে থাকবে।

নতুন Sample-এর জন্য Prediction

নতুন Sample $X = 5$ -এর জন্য Ridge Model-এর Prediction:

$$\hat{y} = 0.5000 \times 5 + 2.5000 = 2.5000 + 2.5000 = 5.0000$$

সুতরাং $X = 5$ -এর জন্য Ridge Regression-এর Prediction হলো $\hat{y} = 5.0000$ ।

Chapter 5 — Lasso Regression

1. Introduction

Lasso Regression (Least Absolute Shrinkage and Selection Operator) হলো Linear Regression-এর আরেকটি উন্নত সংস্করণ, যেখানে L1 Regularization ব্যবহার করা হয়। Ridge-এর মতো এটিও Overfitting কমায়, কিন্তু এর একটি বিশেষ ক্ষমতা আছে — এটি অপ্রয়োজনীয় Feature-এর Weight একেবারে শূন্য করে দিতে পারে।

2. Why This Algorithm Was Developed

Ridge Regression Overfitting ও Multicollinearity সমাধান করলেও, এটি সব Feature Model-এ রেখে দেয় — এমনকি অপ্রয়োজনীয়গুলোও। High-dimensional Dataset-এ (যেখানে শত শত বা হাজারো Feature থাকতে পারে) এটি Model-কে অপ্রয়োজনীয়ভাবে জটিল রাখে। Robert Tibshirani ১৯৯৬ সালে Lasso প্রস্তাব করেন, যা Regularization-এর পাশাপাশি স্বয়ংক্রিয় Feature Selection-ও করতে পারে — একই সাথে দুটি সমস্যার সমাধান।

3. Real-life Motivation

যেসব Dataset-এ অনেক Feature থাকে এবং ধারণা করা হয় সব Feature গুরুত্বপূর্ণ নয়, সেখানে Lasso সবচেয়ে কার্যকর — যেমন Text Data-তে হাজারো Word-Feature থাকা Sentiment Analysis, অথবা Genomic Data-তে হাজারো Gene থেকে গুরুত্বপূর্ণ কয়েকটি খুঁজে বের করা।

4. Intuition

Ridge Weight-কে ছোট করে কিন্তু কখনো শূন্যে পৌঁছায় না — অনেকটা ব্রেক চেপে গাড়ি ধীর করার মতো। Lasso এর চেয়ে কঠোর — এটি অপ্রয়োজনীয় Feature-এর Weight একেবারে শূন্যে নামিয়ে সেটিকে Model থেকে বাদ দিয়ে দেয়, যেন Model নিজেই সিদ্ধান্ত নিচ্ছে কোন Feature রাখা দরকার আর কোনটা নয়।

5. Working Principle

L1 Regularization-এ Loss Function-এর সাথে সকল Weight-এর Absolute Value ($|w|$) যোগ করা হয়। ফলে Model চেষ্টা করে অপ্রয়োজনীয় Feature-এর Weight একেবারে 0 করে দিতে। অর্থাৎ যেসব Feature Prediction-এর জন্য তেমন গুরুত্বপূর্ণ নয়, সেগুলো Model নিজেই বাদ দেয়। এজন্য L1 Regularization-কে Feature Selection Technique-ও বলা হয়।

6. Mathematical Backend

$$J(W,b) = \text{MSE} + \lambda \sum_i |w_i|$$

Ridge-এর w^2 Penalty থেকে ভিন্ন, $|w|$ Penalty-এর একটি বিশেষ গাণিতিক বৈশিষ্ট্য আছে — এর Derivative $w = 0$ -এর কাছে অবিচ্ছিন্ন (constant) থাকে, ফলে Optimization Process Weight-কে ঠিক শূন্যে ঠেলে দিতে পারে (Ridge-এ যা সম্ভব হয় না)।

7. Formula

$$\text{Loss} = \text{MSE} + \lambda \sum_i |w_i|$$

8. Formula Derivation

$|w|$ ফাংশনটি $w = 0$ -এ Differentiable নয়, তাই Lasso-এর জন্য OLS বা Ridge-এর মতো একটি সরল Closed-form সমাধান নেই। এর পরিবর্তে Coordinate Descent বা Subgradient পদ্ধতি ব্যবহার করে সমাধান বের করা হয়, যেখানে প্রতিটি Weight পর্যায়ক্রমে Update করা হয় এবং একটি Soft-Thresholding Function প্রয়োগ করা হয়:

$$w_i \leftarrow \text{sign}(w_i) \cdot \max(|w_i| - \lambda, 0)$$

এই Soft-Thresholding সূত্রটিই মূল কারণ কেন ছোট Weight একেবারে শূন্যে নেমে যায় — যদি $|w_i| \leq \lambda$ হয়, তাহলে w_i সরাসরি 0 হয়ে যায়।

9. Meaning of Every Symbol

Symbol	Meaning
MSE	Mean Squared Error
λ (alpha)	Regularization Strength
$ w_i $	প্রতিটি Weight-এর Absolute Value
$\text{sign}(w_i)$	w_i -এর চিহ্ন (+1 বা -1)

10. Step-by-Step Formula Explanation

- মূল MSE গণনা করা হয়।
- প্রতিটি Weight-এর Absolute Value যোগ করে Penalty তৈরি করা হয়।
- Coordinate Descent-এর মাধ্যমে একটি একটি করে Weight Update করা হয়, প্রতিবার বাকি Weight স্থির রেখে।
- Soft-Thresholding প্রয়োগ করে ছোট Weight-কে শূন্যে নামিয়ে দেওয়া হয়।
- সব Weight স্থিতিশীল না হওয়া পর্যন্ত এই প্রক্রিয়া পুনরাবৃত্তি হয়।

11. Numerical Example

ধরো একটি Model Train করার পরে (Regularization ছাড়া) Weight পাওয়া গেল:

Feature	OLS Weight	Lasso Weight (λ যথেষ্ট বড়)
Age	18	18
Salary	32	0
Balance	25	22
Gender	6	0
City	4	3

এখানে Salary এবং Gender-এর Weight 0 হয়ে গেছে। অর্থাৎ Model সিদ্ধান্ত নিয়েছে এই দুটি Feature Prediction-এর জন্য প্রয়োজনীয় নয় — ভবিষ্যতে Prediction করার সময় এই Feature দুটি আর ব্যবহার হবে না।

12. Visualization

all weights $\rightarrow$ apply soft-threshold(w, λ) $\rightarrow$ small weights snap to 0 (dropped features) $\rightarrow$ remaining nonzero weights = selected features

13. Hyperparameters

Parameter	Default	ব্যাখ্যা
alpha	1.0	Regularization Strength। বড় alpha মানে বেশি Feature বাদ পড়বে (Sparser Model); খুব বেশি হলে সব Weight শূন্য হয়ে Underfitting হতে পারে।
fit_intercept	True	Bias Term শেখা হবে কিনা।
max_iter	1000	Coordinate Descent-এর সর্বোচ্চ Iteration সংখ্যা।
tol	1e-4	Convergence Threshold।
selection	'cyclic'	'random' করলে প্রতিটি Iteration-এ দৈবভাবে একটি Coefficient বেছে Update করা হয়, যা কখনো কখনো দ্রুত Converge করে।
warm_start	False	True করলে আগের fit()-এর সমাধান থেকে পরবর্তী Training শুরু হয়।

PRACTICAL TIP

Ridge-এর মতো Lasso-তেও Feature Scaling অত্যাবশ্যক — একই কারণে, কারণ Penalty Term সরাসরি Weight-এর Magnitude-এর ওপর নির্ভর করে।

14. Scikit-learn Import

```
from sklearn.linear_model import Lasso
```

15. Python Example

```
from sklearn.linear_model import Lasso
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

model = make_pipeline(StandardScaler(), Lasso(alpha=0.1))

model.fit(X_train, y_train)

predictions = model.predict(X_test)

print(model[-1].coef_) # কিছু কোয়েফিসিয়েন্ট ঠিক 0.0 হবে
```

16. Prediction Process

Training শেষে `coef_`-একিছু মান ঠিক 0.0 থাকবে — এগুলোই বাদ পড়া Feature। প্রেডিকশন এখনও একই সূত্রে হয়: $\hat{y} = XW + b$, কিন্তু এখন W -এর অনেক Component শূন্য হওয়ায় কার্যত কম Feature ব্যবহার করেই প্রেডিকশন হয়, যা Model-কে সরল ও দ্রুত করে তোলে।

17. Advantages

- স্বয়ংক্রিয় Feature Selection করে — Model নিজেই অপয়োজনীয় Feature বাদ দেয়।
- ফলে Model বেশি Interpretable ও Sparse হয়।
- High-dimensional Dataset-এ (Feature সংখ্যা > Sample সংখ্যা) কার্যকর।

18. Disadvantages

- যখন একাধিক Feature পরস্পর অত্যন্ত সম্পর্কযুক্ত, তখন Lasso দৈবভাবে যেকোনো একটিকে বেছে নেয় এবং বাকিদের বাদ দেয় — যা সবসময় কাম্য নয়।
- Ridge-এর তুলনায় Coefficient Estimation কম স্থিতিশীল হতে পারে।
- Closed-form সমাধান নেই, তাই Iterative Optimization প্রয়োজন।

19. Applications

- Text Mining ও NLP-তে হাজারো Word-Feature থেকে গুরুত্বপূর্ণগুলো বেছে নেওয়া।
- Genomic Data Analysis-এ হাজারো Gene থেকে প্রাসঙ্গিক কয়েকটি চিহ্নিত করা।

- যেকোনো High-dimensional সমস্যায় যেখানে Model Simplicity ও Interpretability গুরুত্বপূর্ণ।

20. Comparison with Previous Algorithm

দিক	Ridge Regression (L2)	Lasso Regression (L1)
Penalty	$\sum w_i^2$ (Square)	$\sum w_i $ (Absolute Value)
Feature Selection	না	হ্যাঁ (Weight = 0 করে)
Closed-form Solution	আছে	নেই (Iterative প্রয়োজন)
Correlated Feature	সবগুলো রাখে, Weight ভাগ করে	একটি রাখে, বাকিগুলো বাদ দেয়
উপযুক্ত যখন	সব Feature গুরুত্বপূর্ণ	অনেক Feature অপ্রয়োজনীয়

21. Common Mistakes

- Feature Scaling ছাড়া Lasso প্রয়োগ করা।
- খুব বড় alpha সেট করে সব Weight শূন্য করে ফেলা (সম্পূর্ণ Underfitting)।
- Correlated Feature-এর ক্ষেত্রে Lasso-এর Random Selection Behavior না বোঝা।

22. Interview Questions

Q: কেন L1 Penalty Weight-কে শূন্য করতে পারে কিন্তু L2 পারে না?

A: $|w|$ -এর Derivative $w=0$ -এর কাছে একটি Constant মান ($\pm\lambda$) থেকে যায়, তাই Optimization প্রক্রিয়া Weight-কে সম্পূর্ণ শূন্যে ঠেলে দিতে পারে। কিন্তু w^2 -এর Derivative হলো $2w$, যা w শূন্যের কাছে যাওয়ার সাথে সাথে নিজেও ছোট হয়ে যায় — তাই কখনো Weight-কে ঠিক শূন্যে পৌঁছাতে বাধ্য করতে পারে না।

Q: Lasso-কে কেন 'Feature Selection Technique' বলা হয়?

A: কারণ এটি Training-এর সময় নিজে থেকেই অপ্রয়োজনীয় Feature-এর Weight শূন্য করে দেয়, ফলে চূড়ান্ত Model-এ শুধুমাত্র Nonzero Weight-এর Feature-গুলোই ব্যবহৃত হয়।

23. Summary

Category	Key Points
Key Formula	$\text{Loss} = \text{MSE} + \lambda \sum w_i $
Solution Method	Coordinate Descent + Soft-Thresholding (কোনো Closed-form নেই)
Effect on Weights	অপ্রয়োজনীয় Feature-এর Weight ঠিক 0 হয়ে যায়
Key Ability	Automatic Feature Selection
Exam Focus	L1 বনাম L2 Penalty-এর Derivative-এর পার্থক্য
Interview Focus	Lasso কীভাবে ও কেন Sparse Model তৈরি করে

পরবর্তী অধ্যায়ে: Ridge সব Feature রাখে আর স্থিতিশীলতা দেয়; Lasso Feature বাদ দিয়ে সরলতা দেয়। কিন্তু বাস্তব Dataset-এ প্রায়ই দুটোই একসাথে দরকার হয় — Multicollinearity নিয়ন্ত্রণ এবং Feature Selection। Chapter 6-এ আমরা শিখব Elastic Net, যা Ridge ও Lasso-এর শক্তি একত্রিত করে।

24. Complete Mathematical Implementation (Full Manual Walkthrough)

Lasso Regression সাধারণত Coordinate Descent দিয়ে Train করা হয় — এখানে প্রতিটি Weight-কে একবারে একটি করে Update করে সম্পূর্ণ প্রক্রিয়াটি Manual-ভাবে দেখানো হবে।

Dataset

Manual গণনার সুবিধার জন্য একটি ছোট, ২-Feature Dataset নেওয়া হলো:

Sample (i)	X1	X2	y (Actual)
1	1	3	9
2	2	1	6
3	3	5	15
4	4	2	10

Loss Function ও Coordinate Descent Formula

$$J_{\text{lasso}}(w_1, w_2, b) = (1/2n) \sum_i (\hat{y}_i - y_i)^2 + \lambda \cdot (|w_1| + |w_2|)$$

L1 Penalty ($|w|$) Absolute Value Function হওয়ায় এর কোনো এক-বিন্দুতে Derivative নেই ($w=0$ -তে একটি Kink আছে), তাই Gradient Descent-এর বদলে Coordinate Descent এবং Soft-Thresholding ব্যবহার করা হয়। প্রতিটি Weight w_j -এর জন্য প্রথমে বাকি সব Weight স্থির রেখে একটি Correlation-জাতীয় রাশি ρ_j (rho) বের করা হয়:

$$\rho_j = \sum_i X_{ij} \cdot (y_i - \hat{y}_i^{(-j)}) \quad [\hat{y}_i^{(-j)} = w_j \text{ ছাড়া বাকি সব Term দিয়ে করা Prediction}]$$

তারপর এই ρ_j -এর ওপর Soft-Thresholding প্রয়োগ করে নতুন Weight পাওয়া যায়:

$$w_j = \text{SoftThreshold}(\rho_j, n \cdot \lambda) / z_j \quad [z_j = \sum_i X_{ij}^2]$$

SoftThreshold(ρ, t) Function-টির কাজ হলো — যদি $|\rho|$ থ্রেশহোল্ড t -এর চেয়ে ছোট হয়, তাহলে সরাসরি Weight-কে শূন্য করে দেওয়া (এখান থেকেই Lasso-এর Feature Selection ক্ষমতা আসে); নাহলে ρ থেকে t বিয়োগ (বা যোগ, ঋণাত্মক হলে) করে একটি ছোট Weight বের করা:

$$\text{SoftThreshold}(\rho, t) = \rho - t \quad (\text{যদি } \rho > t) \quad = \rho + t \quad (\text{যদি } \rho < -t) \quad = 0 \quad (\text{অন্যথায়})$$

এই Dataset-এ $z_1 = \sum X_1^2 = 1+4+9+16 = 30.0$, $z_2 = \sum X_2^2 = 9+1+25+4 = 39.0$, এবং $\lambda = 1.5$ বাছাই করা হলো, ফলে থ্রেশহোল্ড $t = n \cdot \lambda = 4 \times 1.5 = 6.0$ ।

Bias b Regularize করা হয় না, তাই তা প্রতি Round-এ সরাসরি Residual-এর গড় হিসেবে বের করা হয়:

$$b = \text{mean}(y - w_1 \cdot X_1 - w_2 \cdot X_2)$$

Coordinate Descent — Round-by-Round Manual Calculation

Round 1 (শুরুর $w_1 = 0.0000$, $w_2 = 0.0000$)

প্রথমে Bias আপডেট করা হলো — বর্তমান w_1 , w_2 দিয়ে Residual-এর গড় নিয়ে:

$$b = \text{mean}(y - 0.0000 \cdot X_1 - 0.0000 \cdot X_2) = 10.0000$$

w_1 আপডেট: w_2 ও b স্থির রেখে w_1 -এর জন্য ρ_1 বের করা হলো:

$$\rho_1 = \sum X_{1i} \cdot (y_i - w_2 \cdot X_{2i} - b) = 6.0000$$

যেহেতু $|\rho_1| = 6.0000$ থ্রেশহোল্ড 6.0-এর চেয়ে ছোট, তাই SoftThreshold Function w_1 -কে সরাসরি শূন্য করে দেয়:

$$w_{1_new} = \text{SoftThreshold}(6.0000, 6.0) / 30.0 = 0 / 30.0 = 0.0000$$

w_2 আপডেট: এবার নতুন w_1 ও b স্থির রেখে w_2 -এর জন্য ρ_2 বের করা হলো:

$$\rho_2 = \sum X_{2i} \cdot (y_i - w_1 \cdot X_{1i} - b) = 18.0000$$

যেহেতু $|\rho_2| = 18.0000$ থ্রেশহোল্ড 6.0-এর চেয়ে অনেক বড়, তাই SoftThreshold তা থেকে থ্রেশহোল্ড বিয়োগ করে একটি Non-zero Weight দেয়:

$$w_{2_new} = \text{SoftThreshold}(18.0000, 6.0) / 39.0 = (18.0000 - 6.0) / 39.0 = 0.3077$$

এই Round শেষে সম্পূর্ণ Loss (MSE + Penalty):

$$J = 4.7885$$

Round 2 (শুরুর $w_1 = 0.0000$, $w_2 = 0.3077$)

প্রথমে Bias আপডেট করা হলো — বর্তমান w_1 , w_2 দিয়ে Residual-এর গড় নিয়ে:

$$b = \text{mean}(y - 0.0000 \cdot X_1 - 0.3077 \cdot X_2) = 9.1538$$

w_1 আপডেট: w_2 ও b স্থির রেখে w_1 -এর জন্য ρ_1 বের করা হলো:

$$\rho_1 = \sum X_{1i} \cdot (y_i - w_2 \cdot X_{2i} - b) = 5.8462$$

যেহেতু $|\rho_1| = 5.8462$ থ্রেশহোল্ড 6.0-এর চেয়ে ছোট, তাই SoftThreshold Function w_1 -কে সরাসরি শূন্য করে দেয়:

$$w_{1_new} = \text{SoftThreshold}(5.8462, 6.0) / 30.0 = 0 / 30.0 = 0.0000$$

w2 আপডেট: এবার নতুন w1 ও b স্থির রেখে w2-এর জন্য p2 বের করা হলো:

$$p2 = \sum X2_i \cdot (y_i - w1 \cdot X1_i - b) = 27.3077$$

যেহেতু $|p2| = 27.3077$ থ্রেশহোল্ড 6.0-এর চেয়ে অনেক বড়, তাই SoftThreshold তা থেকে থ্রেশহোল্ড বিয়োগ করে একটি Non-zero Weight দেয়:

$$w2_new = \text{SoftThreshold}(27.3077, 6.0) / 39.0 = (27.3077 - 6.0) / 39.0 = 0.5464$$

এই Round শেষে সম্পূর্ণ Loss (MSE + Penalty):

$$J = 4.1528$$

Round 3 (শুরু w1 = 0.0000, w2 = 0.5464)

প্রথমে Bias আপডেট করা হলো — বর্তমান w1, w2 দিয়ে Residual-এর গড় নিয়ে:

$$b = \text{mean}(y - 0.0000 \cdot X1 - 0.5464 \cdot X2) = 8.4975$$

w1 আপডেট: w2 ও b স্থির রেখে w1-এর জন্য p1 বের করা হলো:

$$p1 = \sum X1_i \cdot (y_i - w2 \cdot X2_i - b) = 5.7268$$

যেহেতু $|p1| = 5.7268$ থ্রেশহোল্ড 6.0-এর চেয়ে ছোট, তাই SoftThreshold Function w1-কে সরাসরি শূন্য করে দেয়:

$$w1_new = \text{SoftThreshold}(5.7268, 6.0) / 30.0 = 0 / 30.0 = 0.0000$$

w2 আপডেট: এবার নতুন w1 ও b স্থির রেখে w2-এর জন্য p2 বের করা হলো:

$$p2 = \sum X2_i \cdot (y_i - w1 \cdot X1_i - b) = 34.5271$$

যেহেতু $|p2| = 34.5271$ থ্রেশহোল্ড 6.0-এর চেয়ে অনেক বড়, তাই SoftThreshold তা থেকে থ্রেশহোল্ড বিয়োগ করে একটি Non-zero Weight দেয়:

$$w2_new = \text{SoftThreshold}(34.5271, 6.0) / 39.0 = (34.5271 - 6.0) / 39.0 = 0.7315$$

এই Round শেষে সম্পূর্ণ Loss (MSE + Penalty):

$$J = 3.7704$$

Round 4 (শুরু w1 = 0.0000, w2 = 0.7315)

প্রথমে Bias আপডেট করা হলো — বর্তমান w1, w2 দিয়ে Residual-এর গড় নিয়ে:

$$b = \text{mean}(y - 0.0000 \cdot X1 - 0.7315 \cdot X2) = 7.9885$$

w1 আপডেট: w2 ও b স্থির রেখে w1-এর জন্য p1 বের করা হলো:

$$p1 = \sum X1_i \cdot (y_i - w2 \cdot X2_i - b) = 5.6343$$

যেহেতু $|p1| = 5.6343$ থ্রেশহোল্ড 6.0-এর চেয়ে ছোট, তাই SoftThreshold Function $w1$ -কে সরাসরি শূন্য করে দেয়:

$$w1_new = \text{SoftThreshold}(5.6343, 6.0) / 30.0 = 0 / 30.0 = 0.0000$$

$w2$ আপডেট: এবার নতুন $w1$ ও b স্থির রেখে $w2$ -এর জন্য $p2$ বের করা হলো:

$$p2 = \sum X2_i \cdot (y_i - w1 \cdot X1_i - b) = 40.1268$$

যেহেতু $|p2| = 40.1268$ থ্রেশহোল্ড 6.0-এর চেয়ে অনেক বড়, তাই SoftThreshold তা থেকে থ্রেশহোল্ড বিয়োগ করে একটি Non-zero Weight দেয়:

$$w2_new = \text{SoftThreshold}(40.1268, 6.0) / 39.0 = (40.1268 - 6.0) / 39.0 = 0.8750$$

এই Round শেষে সম্পূর্ণ Loss (MSE + Penalty):

$$J = 3.5403$$

Round 5 (শুরু $w1 = 0.0000$, $w2 = 0.8750$)

প্রথমে Bias আপডেট করা হলো — বর্তমান $w1$, $w2$ দিয়ে Residual-এর গড় নিয়ে:

$$b = \text{mean}(y - 0.0000 \cdot X1 - 0.8750 \cdot X2) = 7.5936$$

$w1$ আপডেট: $w2$ ও b স্থির রেখে $w1$ -এর জন্য $p1$ বের করা হলো:

$$p1 = \sum X1_i \cdot (y_i - w2 \cdot X2_i - b) = 5.5625$$

যেহেতু $|p1| = 5.5625$ থ্রেশহোল্ড 6.0-এর চেয়ে ছোট, তাই SoftThreshold Function $w1$ -কে সরাসরি শূন্য করে দেয়:

$$w1_new = \text{SoftThreshold}(5.5625, 6.0) / 30.0 = 0 / 30.0 = 0.0000$$

$w2$ আপডেট: এবার নতুন $w1$ ও b স্থির রেখে $w2$ -এর জন্য $p2$ বের করা হলো:

$$p2 = \sum X2_i \cdot (y_i - w1 \cdot X1_i - b) = 44.4701$$

যেহেতু $|p2| = 44.4701$ থ্রেশহোল্ড 6.0-এর চেয়ে অনেক বড়, তাই SoftThreshold তা থেকে থ্রেশহোল্ড বিয়োগ করে একটি Non-zero Weight দেয়:

$$w2_new = \text{SoftThreshold}(44.4701, 6.0) / 39.0 = (44.4701 - 6.0) / 39.0 = 0.9864$$

এই Round শেষে সম্পূর্ণ Loss (MSE + Penalty):

$$J = 3.4019$$

Round 6 (শুরু $w1 = 0.0000$, $w2 = 0.9864$)

প্রথমে Bias আপডেট করা হলো — বর্তমান $w1$, $w2$ দিয়ে Residual-এর গড় নিয়ে:

$$b = \text{mean}(y - 0.0000 \cdot X_1 - 0.9864 \cdot X_2) = 7.2874$$

w1 আপডেট: w2 ও b স্থির রেখে w1-এর জন্য p1 বের করা হলো:

$$p_1 = \sum X_{1i} \cdot (y_i - w_2 \cdot X_{2i} - b) = 5.5068$$

যেহেতু $|p_1| = 5.5068$ থ্রেশহোল্ড 6.0-এর চেয়ে ছোট, তাই SoftThreshold Function w1-কে সরাসরি শূন্য করে দেয়:

$$w_{1_new} = \text{SoftThreshold}(5.5068, 6.0) / 30.0 = 0 / 30.0 = 0.0000$$

w2 আপডেট: এবার নতুন w1 ও b স্থির রেখে w2-এর জন্য p2 বের করা হলো:

$$p_2 = \sum X_{2i} \cdot (y_i - w_1 \cdot X_{1i} - b) = 47.8390$$

যেহেতু $|p_2| = 47.8390$ থ্রেশহোল্ড 6.0-এর চেয়ে অনেক বড়, তাই SoftThreshold তা থেকে থ্রেশহোল্ড বিয়োগ করে একটি Non-zero Weight দেয়:

$$w_{2_new} = \text{SoftThreshold}(47.8390, 6.0) / 39.0 = (47.8390 - 6.0) / 39.0 = 1.0728$$

এই Round শেষে সম্পূর্ণ Loss (MSE + Penalty):

$$J = 3.3186$$

Round 7 (শুরুর w1 = 0.0000, w2 = 1.0728)

প্রথমে Bias আপডেট করা হলো — বর্তমান w1, w2 দিয়ে Residual-এর গড় নিয়ে:

$$b = \text{mean}(y - 0.0000 \cdot X_1 - 1.0728 \cdot X_2) = 7.0498$$

w1 আপডেট: w2 ও b স্থির রেখে w1-এর জন্য p1 বের করা হলো:

$$p_1 = \sum X_{1i} \cdot (y_i - w_2 \cdot X_{2i} - b) = 5.4636$$

যেহেতু $|p_1| = 5.4636$ থ্রেশহোল্ড 6.0-এর চেয়ে ছোট, তাই SoftThreshold Function w1-কে সরাসরি শূন্য করে দেয়:

$$w_{1_new} = \text{SoftThreshold}(5.4636, 6.0) / 30.0 = 0 / 30.0 = 0.0000$$

w2 আপডেট: এবার নতুন w1 ও b স্থির রেখে w2-এর জন্য p2 বের করা হলো:

$$p_2 = \sum X_{2i} \cdot (y_i - w_1 \cdot X_{1i} - b) = 50.4521$$

যেহেতু $|p_2| = 50.4521$ থ্রেশহোল্ড 6.0-এর চেয়ে অনেক বড়, তাই SoftThreshold তা থেকে থ্রেশহোল্ড বিয়োগ করে একটি Non-zero Weight দেয়:

$$w_{2_new} = \text{SoftThreshold}(50.4521, 6.0) / 39.0 = (50.4521 - 6.0) / 39.0 = 1.1398$$

এই Round শেষে সম্পূর্ণ Loss (MSE + Penalty):

$$J = 3.2685$$

Round 8 (শুরুর $w1 = 0.0000$, $w2 = 1.1398$)

প্রথমে Bias আপডেট করা হলো — বর্তমান $w1$, $w2$ দিয়ে Residual-এর গড় নিয়ে:

$$b = \text{mean}(y - 0.0000 \cdot X1 - 1.1398 \cdot X2) = 6.8656$$

$w1$ আপডেট: $w2$ ও b স্থির রেখে $w1$ -এর জন্য $p1$ বের করা হলো:

$$p1 = \sum X1_i \cdot (y_i - w2 \cdot X2_i - b) = 5.4301$$

যেহেতু $|p1| = 5.4301$ থ্রেশহোল্ড 6.0-এর চেয়ে ছোট, তাই SoftThreshold Function $w1$ -কে সরাসরি শূন্য করে দেয়:

$$w1_{\text{new}} = \text{SoftThreshold}(5.4301, 6.0) / 30.0 = 0 / 30.0 = 0.0000$$

$w2$ আপডেট: এবার নতুন $w1$ ও b স্থির রেখে $w2$ -এর জন্য $p2$ বের করা হলো:

$$p2 = \sum X2_i \cdot (y_i - w1 \cdot X1_i - b) = 52.4788$$

যেহেতু $|p2| = 52.4788$ থ্রেশহোল্ড 6.0-এর চেয়ে অনেক বড়, তাই SoftThreshold তা থেকে থ্রেশহোল্ড বিয়োগ করে একটি Non-zero Weight দেয়:

$$w2_{\text{new}} = \text{SoftThreshold}(52.4788, 6.0) / 39.0 = (52.4788 - 6.0) / 39.0 = 1.1918$$

এই Round শেষে সম্পূর্ণ Loss (MSE + Penalty):

$$J = 3.2384$$

Training Progress Table

Round	w1	w2	Bias	Loss
1	0.0000	0.3077	10.0000	4.7885
2	0.0000	0.5464	9.1538	4.1528
3	0.0000	0.7315	8.4975	3.7704
4	0.0000	0.8750	7.9885	3.5403
5	0.0000	0.9864	7.5936	3.4019
6	0.0000	1.0728	7.2874	3.3186
7	0.0000	1.1398	7.0498	3.2685
8	0.0000	1.1918	6.8656	3.2384

শেষে কোন Weight শূন্য হলো — Mathematical ব্যাখ্যা

সব Round জুড়ে $|p1|$ সবসময় থ্রেশহোল্ড 6.0-এর চেয়ে ছোট থেকেছে (সর্বোচ্চ 6.0000), তাই SoftThreshold Function প্রতিবারই $w1$ -কে শূন্যে নামিয়ে দিয়েছে। এর কারণ হলো $X1$ এবং y -এর মধ্যে সম্পর্ক ($w2$, b বিয়োগ করার পরেও) যথেষ্ট শক্তিশালী নয় যে তা Penalty অতিক্রম করে যেতে পারে। ফলে $w1 = 0$ — অর্থাৎ Lasso স্বয়ংক্রিয়ভাবে $X1$ Feature-টিকে Model থেকে বাদ দিয়ে দিয়েছে (Automatic Feature Selection), অন্যদিকে $w2$ বারবার থ্রেশহোল্ড অতিক্রম করায় ধীরে ধীরে বড় হয়ে 1.1918-এ পৌঁছেছে।

Final Learned Equation

$$w1 = 0.0000, w2 = 1.1918, b = 6.8656$$

$$\hat{y} = 0.0000 \cdot X1 + 1.1918 \cdot X2 + 6.8656$$

নতুন Sample-এর জন্য Prediction

নতুন Sample $X1 = 5$, $X2 = 3$ -এর জন্য Prediction:

$$\hat{y} = 0.0000 \times 5 + 1.1918 \times 3 + 6.8656 = 0.0000 + 3.5753 + 6.8656 = 10.4409$$

সুতরাং $X1 = 5$, $X2 = 3$ -এর জন্য Lasso Model-এর Prediction হলো $\hat{y} \approx 10.4409$ । লক্ষ্যণীয় — $w1$ শূন্য হওয়ায় $X1$ -এর মান Prediction-এ কোনো ভূমিকা রাখছে না।

Chapter 6 — Elastic Net Regression

1. Introduction

Elastic Net হলো Ridge এবং Lasso-এর সমন্বিত সংস্করণ। এতে L1 এবং L2 উভয় ধরনের Regularization একসাথে ব্যবহার করা হয়। ফলে কিছু Feature-এর Weight Zero হয়ে যায় (Lasso-এর মতো) এবং বাকি Weight-গুলো ছোট হয়ে যায় (Ridge-এর মতো)। এটি Feature Selection এবং Multicollinearity — উভয় সমস্যাই একসাথে সমাধান করতে পারে।

2. Why This Algorithm Was Developed

Lasso-এর একটি সীমাবদ্ধতা হলো — যখন একাধিক Feature পরস্পর অত্যন্ত সম্পর্কযুক্ত (Correlated), তখন এটি দৈবভাবে একটিকে বেছে নেয় ও বাকিদের সম্পূর্ণ বাদ দিয়ে দেয়, যা তথ্য হারানোর ঝুঁকি তৈরি করে। এছাড়া Feature সংখ্যা Sample সংখ্যার চেয়ে বেশি হলে Lasso একসাথে সর্বোচ্চ n -সংখ্যক Feature-ই বেছে নিতে পারে। Hui Zou ও Trevor Hastie ২০০৫ সালে Elastic Net প্রস্তাব করেন, যা L1 ও L2 উভয় Penalty একত্রিত করে এই সীমাবদ্ধতাগুলো দূর করে।

3. Real-life Motivation

যেসব Dataset-এ অনেক Feature আছে এবং সেগুলোর মধ্যে অনেকগুলো একে অপরের সাথে Correlated, সেখানে Elastic Net সবচেয়ে কার্যকর — যেমন Genomics-এ হাজারো পরস্পর-সম্পর্কিত Gene-Feature, অথবা বড় ও জটিল Marketing Dataset যেখানে একসাথে Feature Selection ও Overfitting নিয়ন্ত্রণ দুটোই প্রয়োজন।

4. Intuition

Elastic Net-কে Ridge ও Lasso-এর মধ্যে একটি "স্লাইডার" হিসেবে কল্পনা করা যায়। $l1_ratio$ নামক একটি প্যারামিটার দিয়ে ঠিক করা যায় Model কতটা Lasso-এর মতো (Feature বাদ দেবে) আর কতটা Ridge-এর মতো (Weight ছোট করবে) আচরণ করবে।

5. Working Principle

Elastic Net L1 এবং L2 উভয় Penalty Term একসাথে Loss Function-এ যোগ করে। ফলাফল হিসেবে Model একইসাথে কিছু Feature-এর Weight শূন্য করে (Feature Selection) এবং বাকি সব Feature-এর Weight ছোট করে (Shrinkage)।

6. Mathematical Backend

$$J(W,b) = \text{MSE} + \lambda_1 \sum_i |w_i| + \lambda_2 \sum_i w_i^2$$

7. Formula

$$\text{Loss} = \text{MSE} + \lambda_1 \sum_i |w_i| + \lambda_2 \sum_i w_i^2$$

8. Formula Derivation

Scikit-Learn-এ এই দুই Penalty-কে একটি একক alpha (মোট Regularization Strength) ও l1_ratio (L1-এর অংশ) দিয়ে Parameterize করা হয়:

$$\text{Penalty} = \text{alpha} \times [\text{l1_ratio} \times \sum |w_i| + (1 - \text{l1_ratio}) \times \sum w_i^2]$$

l1_ratio = 1 হলে এটি সম্পূর্ণ Lasso-এর সমান হয়ে যায়; l1_ratio = 0 হলে এটি সম্পূর্ণ Ridge-এর সমান হয়ে যায়। Lasso-এর মতোই এখানে |w| Term থাকায় কোনো সরল Closed-form Solution নেই — Coordinate Descent ব্যবহার করে সমাধান বের করা হয়।

9. Meaning of Every Symbol

Symbol	Meaning
λ_1	L1 Regularization-এর Penalty Strength
λ_2	L2 Regularization-এর Penalty Strength
alpha	মোট Regularization Strength (sklearn)
l1_ratio	মোট Penalty-এর মধ্যে L1-এর অংশ (0 থেকে 1)

10. Step-by-Step Formula Explanation

- মূল MSE গণনা করা হয়।
- L1 Penalty ($\sum |w_i|$) এবং L2 Penalty ($\sum w_i^2$) আলাদাভাবে গণনা করা হয়।
- l1_ratio দিয়ে দুটি Penalty-কে ওজন দিয়ে একত্রিত করা হয়।
- Coordinate Descent ব্যবহার করে Weight ধাপে ধাপে Update করা হয়, একই সাথে কিছু Weight শূন্যে নেমে আসে এবং বাকিগুলো ছোট হয়।

11. Numerical Example

ধরো Linear Regression-এর (Regularization ছাড়া) Weight ছিল:

Feature	OLS Weight	Elastic Net Weight
Age	25	18
Salary	40	20
Balance	18	12
Gender	8	0

Feature	OLS Weight	Elastic Net Weight
City	5	2

এখানে Gender-এর Weight Zero হয়ে গেছে (Lasso-এর প্রভাব) এবং বাকি সব Weight ছোট হয়েছে (Ridge-এর প্রভাব)। অর্থাৎ Model একই সঙ্গে Feature Selection এবং Weight Shrinkage করেছে।

12. Visualization

$l1_ratio = 0 \rightarrow$ pure Ridge (no zeros) $l1_ratio = 1 \rightarrow$ pure Lasso (many zeros) $0 < l1_ratio < 1 \rightarrow$ Elastic Net (some zeros, rest shrunk)

13. Hyperparameters

Parameter	Default	ব্যাখ্যা
alpha	1.0	মোট Regularization Strength — L1 ও L2 উভয় Penalty-এর সম্মিলিত শক্তি নিয়ন্ত্রণ করে।
l1_ratio	0.5	L1 ও L2-এর মধ্যে ভারসাম্য। 0 = সম্পূর্ণ Ridge, 1 = সম্পূর্ণ Lasso।
fit_intercept	True	Bias Term শেখা হবে কিনা।
max_iter	1000	Coordinate Descent-এর সর্বোচ্চ Iteration।
tol	1e-4	Convergence Threshold।
selection	'cyclic'	'random' করলে দৈবভাবে Coefficient বেছে Update করা হয়।

PRACTICAL TIP

$l1_ratio$ ও α দুটোই সাধারণত ElasticNetCV ব্যবহার করে Cross-Validation-এর মাধ্যমে নির্ধারণ করা হয়, যাতে সর্বোত্তম সমন্বয় স্বয়ংক্রিয়ভাবে খুঁজে পাওয়া যায়।

14. Scikit-learn Import

```
from sklearn.linear_model import ElasticNet, ElasticNetCV
```

15. Python Example

```
from sklearn.linear_model import ElasticNet
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

model = make_pipeline(StandardScaler(), ElasticNet(alpha=0.1, l1_ratio=0.5))

model.fit(X_train, y_train)

predictions = model.predict(X_test)

print(model[-1].coef_)
```

16. Prediction Process

Training শেষে `coef_`-এ কিছু মান ঠিক 0.0 এবং বাকি সব Ridge-এর মতো ছোট মান থাকবে। প্রেডিকশন একই $\hat{y} = XW + b$ সূত্রে হয়, যেখানে W -এ Lasso ও Ridge উভয়ের প্রভাব মিশ্রিত থাকে।

17. Advantages

- Feature Selection এবং Multicollinearity নিয়ন্ত্রণ — উভয়ের সুবিধা একসাথে পাওয়া যায়।
- Correlated Feature Group-কে একসাথে রাখতে বা বাদ দিতে সক্ষম (Lasso-এর দৈব আচরণের চেয়ে বেশি স্থিতিশীল)।
- Feature সংখ্যা Sample সংখ্যার চেয়ে বেশি হলেও ভালো কাজ করে।

18. Disadvantages

- দুটি Hyperparameter (α , $l1_ratio$) টিউন করতে হয়, যা Tuning প্রক্রিয়াকে জটিল করে তোলে।
- Closed-form সমাধান নেই, তাই Iterative Optimization প্রয়োজন।
- সঠিকভাবে Tune না করলে Ridge বা Lasso-এর মূল সুবিধাগুলোর কোনোটিই পুরোপুরি পাওয়া নাও যেতে পারে।

19. Applications

- Genomics-এ হাজারো পরস্পর-সম্পর্কিত Gene-Feature বিশ্লেষণ।
- বড় ও জটিল Marketing/Financial Dataset যেখানে Feature Selection ও Overfitting নিয়ন্ত্রণ দুটোই দরকার।
- High-dimensional Dataset যেখানে Feature সংখ্যা Sample সংখ্যার চেয়ে বেশি।

20. Comparison with Previous Algorithm

Model	Regularization	Feature Selection	মূল বৈশিষ্ট্য
Linear Regression (OLS)	নেই	না	কোনো Penalty নেই; Overfitting-প্রবণ
Ridge Regression	L2	না	সব Feature রাখে, Weight ছোট করে
Lasso Regression	L1	হ্যাঁ	অপ্রয়োজনীয় Feature বাদ দেয় (Weight = 0)
Elastic Net	L1 + L2	হ্যাঁ (আংশিক)	কিছু Feature বাদ দেয় এবং বাকি Weight-ও ছোট করে

সংক্ষেপে: Linear Regression কোনো Regularization ব্যবহার করে না, Ridge Regression L2 Regularization ব্যবহার করে সব Feature রেখে তাদের Weight ছোট করে, Lasso Regression L1 Regularization ব্যবহার করে অপ্রয়োজনীয় Feature-এর Weight Zero করে Feature Selection করে, এবং Elastic Net L1 ও L2 উভয় Regularization ব্যবহার করে একই সঙ্গে Feature Selection এবং Weight Shrinkage করতে পারে।

21. Common Mistakes

- l1_ratio-কে ডিফল্ট 0.5-এ রেখে দেওয়া, Dataset-এর প্রকৃতি বিবেচনা না করে।
- ElasticNetCV ব্যবহার না করে ম্যানুয়ালি ভুল alpha/l1_ratio বেছে নেওয়া।
- Feature Scaling ছাড়া প্রয়োগ করা।

22. Interview Questions

Q: কখন Elastic Net, শুধু Lasso বা শুধু Ridge-এর চেয়ে ভালো পছন্দ?

A: যখন Feature সংখ্যা অনেক বেশি এবং তাদের মধ্যে উল্লেখযোগ্য Correlation থাকে — এমন পরিস্থিতিতে Lasso দৈবভাবে একটি Feature বেছে নেয় ও বাকিদের বাদ দেয়, কিন্তু Elastic Net-এর L2 অংশ Correlated Feature-দের একসাথে Group করে রাখতে সাহায্য করে, ফলে ফলাফল বেশি স্থিতিশীল হয়।

Q: l1_ratio = 0 এবং l1_ratio = 1 হলে Elastic Net কী হয়ে যায়?

A: l1_ratio = 0 হলে এটি সম্পূর্ণ Ridge Regression-এর সমান হয়ে যায়; l1_ratio = 1 হলে এটি সম্পূর্ণ Lasso Regression-এর সমান হয়ে যায়।

23. Summary

Category	Key Points
Key Formula	$Loss = MSE + \lambda_1 \sum w_i + \lambda_2 \sum w_i^2$
Key Parameters	alpha (মোট শক্তি), l1_ratio (L1/L2 ভারসাম্য)

Category	Key Points
Behavior	কিছু Weight শূন্য হয় (Lasso), বাকি ছোট হয় (Ridge)
Best For	অনেক ও পরস্পর-Correlated Feature সহ Dataset
Exam Focus	l1_ratio-এর দুই প্রান্তিক মান (0 ও 1) কী প্রতিনিধিত্ব করে
Interview Focus	Elastic Net কেন Correlated Feature-এ Lasso-এর চেয়ে স্থিতিশীল

PART SUMMARY

Part 1-এ আমরা Regression Family-এর একটি সম্পূর্ণ পথ অতিক্রম করেছি — একটিমাত্র Feature (Simple Linear Regression) থেকে একাধিক Feature (Multiple Linear Regression), তারপর Closed-form গাণিতিক সমাধান (OLS), এবং শেষে তিন ধরনের Regularization (Ridge, Lasso, Elastic Net) যা Overfitting ও Multicollinearity নিয়ন্ত্রণ করে। এই ভিত্তি Part 2 (Classification)-এ Logistic Regression বোঝার জন্য অত্যন্ত গুরুত্বপূর্ণ, কারণ Logistic Regression মূলত Linear Regression-এরই একটি সম্প্রসারণ, যেখানে আউটপুটকে একটি Probability-তে রূপান্তর করা হয়।

পরবর্তী Part-এ: Part 2-তে আমরা Regression থেকে Classification-এ প্রবেশ করব — যেখানে লক্ষ্য থাকবে Continuous মানের বদলে একটি নির্দিষ্ট Category (যেমন Spam/Not-Spam, Disease/No-Disease) ভবিষ্যদ্বাণী করা। এটি শুরু হবে Logistic Regression দিয়ে, যা এই Part-এর ধারণাগুলোর ওপরই গড়ে উঠবে।

24. Complete Mathematical Implementation (Full Manual Walkthrough)

Elastic Net, Lasso-এর L1 Penalty এবং Ridge-এর L2 Penalty — দুটোকেই একসাথে ব্যবহার করে। এখানে Lasso Chapter-এর একই Dataset ব্যবহার করে সম্পূর্ণ Manual Coordinate Descent দেখানো হবে।

Dataset (Lasso-এর মতো একই Dataset)

Sample (i)	X1	X2	y (Actual)
1	1	3	9
2	2	1	6
3	3	5	15
4	4	2	10

L1 অংশ, L2 অংশ ও Total Penalty

Elastic Net-এর Penalty দুটি ভাগে বিভক্ত — L1 (Absolute Value, যা Feature Selection করে) এবং L2 (Square, যা সব Weight-কে সমানভাবে ছোট করে):

$$L1 \text{ অংশ} = \lambda_1 \cdot (|w_1| + |w_2|)$$

$$L2 \text{ অংশ} = \lambda_2 \cdot (w_1^2 + w_2^2)$$

$$\text{Total Loss: } J = (1/2n) \sum_i (\hat{y}_i - y_i)^2 + \lambda_1 \cdot (|w_1| + |w_2|) + \lambda_2 \cdot (w_1^2 + w_2^2)$$

এই উদাহরণে $\lambda_1 = 1.5$ (Lasso Chapter-এর মতোই) এবং $\lambda_2 = 0.5$ বাছাই করা হলো।

Gradient ও Coordinate Descent Formula

L2 অংশটি সব জায়গায় Differentiable, তাই এটি সরাসরি Denominator-এ যোগ হয়ে যায়; কিন্তু L1 অংশ আগের মতোই Soft-Thresholding দাবি করে। তাই Combined Update Rule হলো:

$$w_j = \text{SoftThreshold}(\rho_j, n \cdot \lambda_1) / (z_j + 2n \cdot \lambda_2)$$

এখানে ρ_j এবং z_j ঠিক Lasso-এর মতোই বের হয়, শুধু Denominator-এ L2-এর জন্য অতিরিক্ত $2n \cdot \lambda_2$ যোগ হয়েছে। $n \cdot \lambda_1 = 4 \times 1.5 = 6.0$, এবং Denominator: $z_1 + 2n \cdot \lambda_2 = 30.0 + 4.0 = 34.0$; $z_2 + 2n \cdot \lambda_2 = 39.0 + 4.0 = 43.0$ ।

Coordinate Descent — Round-by-Round Manual Calculation

Round 1 (শুরুর $w_1 = 0.0000$, $w_2 = 0.0000$)

বর্তমান w_1, w_2 দিয়ে Bias আপডেট:

$$b = \text{mean}(y - 0.0000 \cdot X_1 - 0.0000 \cdot X_2) = 10.0000$$

w_1 -এর জন্য ρ_1 বের করে Soft-Threshold প্রয়োগ, তারপর Denominator ($z_1 + 2n \cdot \lambda_2$) দিয়ে ভাগ:

$$\rho_1 = 6.0000$$

$$w_{1_new} = \text{SoftThreshold}(6.0000, 6.0) / 34.0 = 0 / 34.0 = 0.0000$$

w_2 -এর জন্য নতুন w_1 ও b বসিয়ে ρ_2 বের করে একইভাবে Update:

$$\rho_2 = 18.0000$$

$$w_{2_new} = \text{SoftThreshold}(18.0000, 6.0) / 43.0 = (18.0000 - 6.0) / 43.0 = 0.2791$$

এই Round-এর Total Loss (MSE + L1 + L2):

$$J = 4.3739 + 0.4186 + 0.0389 = 4.8314$$

Round 2 (শুরুর $w_1 = 0.0000$, $w_2 = 0.2791$)

বর্তমান w_1, w_2 দিয়ে Bias আপডেট:

$$b = \text{mean}(y - 0.0000 \cdot X_1 - 0.2791 \cdot X_2) = 9.2326$$

w_1 -এর জন্য ρ_1 বের করে Soft-Threshold প্রয়োগ, তারপর Denominator ($z_1 + 2n \cdot \lambda_2$) দিয়ে ভাগ:

$$\rho_1 = 5.8605$$

$$w_{1_new} = \text{SoftThreshold}(5.8605, 6.0) / 34.0 = 0 / 34.0 = 0.0000$$

w2-এর জন্য নতুন w1 ও b বসিয়ে p2 বের করে একইভাবে Update:

$$p2 = 26.4419$$

$$w2_new = \text{SoftThreshold}(26.4419, 6.0) / 43.0 = (26.4419 - 6.0) / 43.0 = 0.4754$$

এই Round-এর Total Loss (MSE + L1 + L2):

$$J = 3.5037 + 0.7131 + 0.1130 = 4.3297$$

Round 3 (শুরু w1 = 0.0000, w2 = 0.4754)

বর্তমান w1, w2 দিয়ে Bias আপডেট:

$$b = \text{mean}(y - 0.0000 \cdot X1 - 0.4754 \cdot X2) = 8.6927$$

w1-এর জন্য p1 বের করে Soft-Threshold প্রয়োগ, তারপর Denominator ($z1 + 2n \cdot \lambda2$) দিয়ে ভাগ:

$$p1 = 5.7623$$

$$w1_new = \text{SoftThreshold}(5.7623, 6.0) / 34.0 = 0 / 34.0 = 0.0000$$

w2-এর জন্য নতুন w1 ও b বসিয়ে p2 বের করে একইভাবে Update:

$$p2 = 32.3806$$

$$w2_new = \text{SoftThreshold}(32.3806, 6.0) / 43.0 = (32.3806 - 6.0) / 43.0 = 0.6135$$

এই Round-এর Total Loss (MSE + L1 + L2):

$$J = 2.9730 + 0.9203 + 0.1882 = 4.0815$$

Round 4 (শুরু w1 = 0.0000, w2 = 0.6135)

বর্তমান w1, w2 দিয়ে Bias আপডেট:

$$b = \text{mean}(y - 0.0000 \cdot X1 - 0.6135 \cdot X2) = 8.3129$$

w1-এর জন্য p1 বের করে Soft-Threshold প্রয়োগ, তারপর Denominator ($z1 + 2n \cdot \lambda2$) দিয়ে ভাগ:

$$p1 = 5.6932$$

$$w1_new = \text{SoftThreshold}(5.6932, 6.0) / 34.0 = 0 / 34.0 = 0.0000$$

w2-এর জন্য নতুন w1 ও b বসিয়ে p2 বের করে একইভাবে Update:

$$p2 = 36.5585$$

$$w2_new = \text{SoftThreshold}(36.5585, 6.0) / 43.0 = (36.5585 - 6.0) / 43.0 = 0.7107$$

এই Round-এর Total Loss (MSE + L1 + L2):

$$J = 2.6401 + 1.0660 + 0.2525 = 3.9586$$

Round 5 (শুরুর $w1 = 0.0000$, $w2 = 0.7107$)

বর্তমান $w1$, $w2$ দিয়ে Bias আপডেট:

$$b = \text{mean}(y - 0.0000 \cdot X1 - 0.7107 \cdot X2) = 8.0457$$

$w1$ -এর জন্য $p1$ বের করে Soft-Threshold প্রয়োগ, তারপর Denominator ($z1 + 2n \cdot \lambda2$) দিয়ে ভাগ:

$$p1 = 5.6447$$

$$w1_new = \text{SoftThreshold}(5.6447, 6.0) / 34.0 = 0 / 34.0 = 0.0000$$

$w2$ -এর জন্য নতুন $w1$ ও b বসিয়ে $p2$ বের করে একইভাবে Update:

$$p2 = 39.4975$$

$$w2_new = \text{SoftThreshold}(39.4975, 6.0) / 43.0 = (39.4975 - 6.0) / 43.0 = 0.7790$$

এই Round-এর Total Loss (MSE + L1 + L2):

$$J = 2.4259 + 1.1685 + 0.3034 = 3.8978$$

Round 6 (শুরুর $w1 = 0.0000$, $w2 = 0.7790$)

বর্তমান $w1$, $w2$ দিয়ে Bias আপডেট:

$$b = \text{mean}(y - 0.0000 \cdot X1 - 0.7790 \cdot X2) = 7.8577$$

$w1$ -এর জন্য $p1$ বের করে Soft-Threshold প্রয়োগ, তারপর Denominator ($z1 + 2n \cdot \lambda2$) দিয়ে ভাগ:

$$p1 = 5.6105$$

$$w1_new = \text{SoftThreshold}(5.6105, 6.0) / 34.0 = 0 / 34.0 = 0.0000$$

$w2$ -এর জন্য নতুন $w1$ ও b বসিয়ে $p2$ বের করে একইভাবে Update:

$$p2 = 41.5651$$

$$w2_new = \text{SoftThreshold}(41.5651, 6.0) / 43.0 = (41.5651 - 6.0) / 43.0 = 0.8271$$

এই Round-এর Total Loss (MSE + L1 + L2):

$$J = 2.2850 + 1.2406 + 0.3420 = 3.8677$$

Round 7 (শুরুর $w1 = 0.0000$, $w2 = 0.8271$)

বর্তমান $w1$, $w2$ দিয়ে Bias আপডেট:

$$b = \text{mean}(y - 0.0000 \cdot X1 - 0.8271 \cdot X2) = 7.7255$$

w1-এর জন্য ρ_1 বের করে Soft-Threshold প্রয়োগ, তারপর Denominator $(z_1 + 2n \cdot \lambda_2)$ দিয়ে ভাগ:

$$\rho_1 = 5.5865$$

$$w1_new = \text{SoftThreshold}(5.5865, 6.0) / 34.0 = 0 / 34.0 = 0.0000$$

w2-এর জন্য নতুন w1 ও b বসিয়ে ρ_2 বের করে একইভাবে Update:

$$\rho_2 = 43.0196$$

$$w2_new = \text{SoftThreshold}(43.0196, 6.0) / 43.0 = (43.0196 - 6.0) / 43.0 = 0.8609$$

এই Round-এর Total Loss (MSE + L1 + L2):

$$J = 2.1909 + 1.2914 + 0.3706 = 3.8528$$

Round 8 (শুরুর w1 = 0.0000, w2 = 0.8609)

বর্তমান w1, w2 দিয়ে Bias আপডেট:

$$b = \text{mean}(y - 0.0000 \cdot X_1 - 0.8609 \cdot X_2) = 7.6325$$

w1-এর জন্য ρ_1 বের করে Soft-Threshold প্রয়োগ, তারপর Denominator $(z_1 + 2n \cdot \lambda_2)$ দিয়ে ভাগ:

$$\rho_1 = 5.5695$$

$$w1_new = \text{SoftThreshold}(5.5695, 6.0) / 34.0 = 0 / 34.0 = 0.0000$$

w2-এর জন্য নতুন w1 ও b বসিয়ে ρ_2 বের করে একইভাবে Update:

$$\rho_2 = 44.0429$$

$$w2_new = \text{SoftThreshold}(44.0429, 6.0) / 43.0 = (44.0429 - 6.0) / 43.0 = 0.8847$$

এই Round-এর Total Loss (MSE + L1 + L2):

$$J = 2.1270 + 1.3271 + 0.3914 = 3.8455$$

Training Progress Table

Round	w1	w2	Bias	Loss
1	0.0000	0.2791	10.0000	4.8314
2	0.0000	0.4754	9.2326	4.3297
3	0.0000	0.6135	8.6927	4.0815
4	0.0000	0.7107	8.3129	3.9586
5	0.0000	0.7790	8.0457	3.8978
6	0.0000	0.8271	7.8577	3.8677

Round	w1	w2	Bias	Loss
7	0.0000	0.8609	7.7255	3.8528
8	0.0000	0.8847	7.6325	3.8455

Final Learned Equation

$$w1 = 0.0000, w2 = 0.8847, b = 7.6325$$

$$\hat{y} = 0.0000 \cdot X1 + 0.8847 \cdot X2 + 7.6325$$

Lasso-তে $w2 = 1.1918$ হয়েছিল, এখানে Elastic Net-এ L2 Penalty-এর কারণে তা আরও একটু কমে $w2 = 0.8847$ -এ নেমেছে — অর্থাৎ L2 অংশ অতিরিক্ত Shrinkage যোগ করেছে, ঠিক যেভাবে Ridge-এ হয়েছিল। একইসাথে $w1$ এখনো শূন্যই থেকেছে, কারণ L1 অংশ এখনো তার Feature Selection ক্ষমতা বজায় রেখেছে।

নতুন Sample-এর জন্য Prediction

নতুন Sample $X1 = 5, X2 = 3$ -এর জন্য Prediction:

$$\hat{y} = 0.0000 \times 5 + 0.8847 \times 3 + 7.6325 = 0.0000 + 2.6542 + 7.6325 = 10.2866$$

সুতরাং $X1 = 5, X2 = 3$ -এর জন্য Elastic Net Model-এর Prediction হলো $\hat{y} \approx 10.2866$ ।